



Sicheres Indoor-Fliegen:

*Eine Einsteigerfreundliche Drohne mit
Kollisionsprävention*

Autor

Kaufmann Cedric G21a
Hubelmatte 26, 6208 Oberkirch

Betreuer

Bucheli Philippe

Maturaarbeit in der Fachschaft
Informatik

Kantonsschule Sursee 24/25



Abstrakt

Das Erlernen des Drohnenfliegens kann am Anfang sehr schwierig und frustrierend sein. Der Drohnenboom der letzten paar Jahre hat immer mehr Menschen dazu gebracht, sich mit der Drohnenfliegen zu befassen. Sei es durch fertig gekaufte Drohnen von DJI oder anderen Herstellern oder gar durch das Bauen einer eigenen Drohne, es passieren immer mehr Unfälle und Kollisionen mit Menschen, Tieren oder Gegenständen. Die Ursache dafür ist häufig, dass besonders am Anfang das Feingefühl für die Steuerung der Drohne fehlt und erlernt werden muss.

Dieser Lernprozess kann mühsam und frustrierend sein, und in den schlimmsten Fällen auch recht kostspielig. Um diesen Lernprozess zu vereinfachen und potenzielle Schäden zu verhindern, werden immer mehr Kollisionspräventionssysteme, welche den angehenden Piloten helfen, in den Drohnen verbaut.

Diese Arbeit ist ein Versuch, ein eigenes Kollisionspräventionssystem zu entwerfen, zu programmieren und zu bauen. Basierend auf einer zuvor selbst gebauten Drohne wurde ein selbst entworfenes System montiert, welches es durch Distanzmessungen ermöglichen sollte, Kollisionen zu erkennen und gegebenenfalls die Flugbahn der Drohne anzupassen.

Inhaltsverzeichnis

Abstrakt	2
Vorwort	5
Programmierung des Mikrocontrollers	6
Verwendete Ressourcen	8
Ultraschallsensor (Modell: Hc-04)	8
1. Allgemeine Informationen über den Sensor	8
2. Aufbau und Funktionsweise	9
3. Programmierung eines HC-sr04 Sensors	10
Time Of Flight Sensor (VL53l0X)	12
1. Allgemeine Informationen über den Sensor	12
2. Programmierung eines VL53l0X Sensors	14
Multiplexer (TCA9548A)	15
1. Allgemeine Informationen über das Modul	15
2. Funktionsweise / Verwendungszweck	15
Mikrocontroller (Teensy 4.1)	16
1. Allgemeine Informationen über den Mikrocontroller	16
2. Verwendung	17
Kommunikationsprotokolle	18
1. Funkprotokoll PPM	18
2. PWM Signale	18
3. PPM Signale	19
Entwicklung der 3d gedruckten Teile	21
1. Erläuterung der verwendeten Software	21
2. Überblick über die entwickelten Bauteile	21
Design der Leiterplatten	28
1. Die wichtigsten Begriffe für das Verständnis	28
Das Ohm'sche Gesetz:	28
Weiter wichtige Begriffe:	29
2. Elektrischer Schaltplan der Leiterplatten	30
3. Probleme und Überlegungen beim Entwickeln der Leiterplatten	31
Code Vertiefung	34
Reflexion	37
Disclaimer	39
Verweise und Links	39

Anhang	40
Abbildungsverzeichnis:	40
Tabellenverzeichnis.....	41
Literaturverzeichnis	41
Deklaration	44

Vorwort

Im Rahmen meiner Maturaarbeit habe ich an einer Drohne gearbeitet, welche ich mit einem Kollisionspräventionssystem ausgestattet habe. Das Grundgerüst meiner gesamten Arbeit ist die Drohne selbst, welche ich zuvor von meinem Onkel als Bausatz geschenkt bekommen habe. Weiter habe ich dann diese Drohne mit dem oben erwähnten System ausgestattet; das System enthält viele verschiedene Komponenten (siehe Verwendete Ressourcen Seite 6), und selbst entwickelte Teile, welche ich mit dem 3D Druckverfahren produziert habe. Um die Drohne nun auch intelligent zu machen, versetzte ich dem eingebauten Mikrocontroller einen Algorithmus, welcher die Signale, basierend auf den Distanzwerten der einzelnen Sensoren, manipuliert.

Es war für mich schon von Anfang an klar, dass ich eine technische Produktion machen werde. Da ich sehr interessiert darin bin, Sachen auch selbst herzustellen und zu entwickeln, hätte mir dieser Aspekt gefehlt, falls ich nur eine wissenschaftliche Arbeit verfasst hätte. Auf das verwendete Thema bin ich nach einem Gespräch mit meinem Onkel gekommen. Die Idee, eine Drohne zu bauen, gefiel mir von Beginn an, da ich mich persönlich sehr für technische Anwendungen, mitunter Drohnen und Roboter, interessiere. Und ich verbringe einen beträchtlichen Teil meiner Freizeit damit, mit verschiedensten Mikrocontrollern kleine Projekte zu entwerfen und zu realisieren. Dieses Interesse und auch mein späterer Studienwunsch Richtung Maschinen- oder Elektroingenieur waren ausschlaggebend für meine Wahl. Dass ich die nötigen Fähigkeiten bis zu einem gewissen Grad schon erlernt hatte, erleichterte mir die Planung der Drohne und der Platinen, sowie der Halterungen für die verschiedensten Sensoren.

Eine gute Vorbereitung und Planung waren für meine Arbeit essenziell, wie ich im weiteren Verlauf der Arbeit immer wieder feststellen durfte. Grundsätzlich bestand meine Organisation darin, mich zuerst vertieft in das Thema einzuarbeiten. In diesem Schritt habe ich mich vermehrt mit verschiedenen Funkprotokollen der RC-Welt auseinandergesetzt. Da ich wusste, dass ich später eines dieser Protokolle verstehen und manipulieren muss. Das Funkprotokoll, welches von der Drohne verwendet wird (wurde von mir entschieden) ist PPM (siehe Seite 18).

Nach der Theorie begann ich damit, die einzelnen Komponenten der Drohne zu entwickeln, primär waren das die Halterungen für die Sensoren. Dies geschah in einem CAD-Programm, namentlich Fusion 360, welches für den privaten Gebrauch kostenlos ist (für genauere Details, wie ich dies getan habe, siehe Seite 20)

Weiter musste die Drohne zusammengebaut und der Mikrocontroller programmiert werden. Die Programmierung des Mikrocontrollers ist im nachfolgenden Kapitel 'Programmierung des Mikrocontrollers' beschrieben. Im Code wurden auch viele Elemente des Funkprotokolls PPM (siehe Seite 18) erarbeitet, welches einen beachtlichen Teil der Zeit in Anspruch nahm. Einen genaueren Überblick über diese Codeabschnitte sind auf Seite: 34 beschrieben.

Programmierung des Mikrocontrollers

Mitunter das wichtigste Element meiner Maturaarbeit ist wohl der Mikrocontroller, welcher zwischen dem Flightcontroller der Drohne und dem RC-Receiver der Drohne sitzt und die Signale, welche der Receiver erhält, abliest und entsprechend den ausgelesenen Daten der verschiedenen Sensoren manipuliert, um das gewünschte Ziel zu erreichen. Verwendet habe ich einen Teensy 4.1 von PJRC (mehr zum Microcontroller siehe Seite 16), welcher relativ einfach mithilfe der Arduino IDE programmiert werden kann. Aus verschiedenen Gründen, wie Leserlichkeit, Autokorrektur, Vorschläge, Plugins und noch weiteren, habe ich mich jedoch dazu entschlossen, nicht die Arduino IDE zu verwenden, sondern eine Visual Studio Code Extension namens PlatformIO, welche grundsätzlich eine Imitation der Arduino IDE ist, aber nun in Visual Studio Code funktioniert.

Das Hauptmenü von PlatformIO sieht so aus:

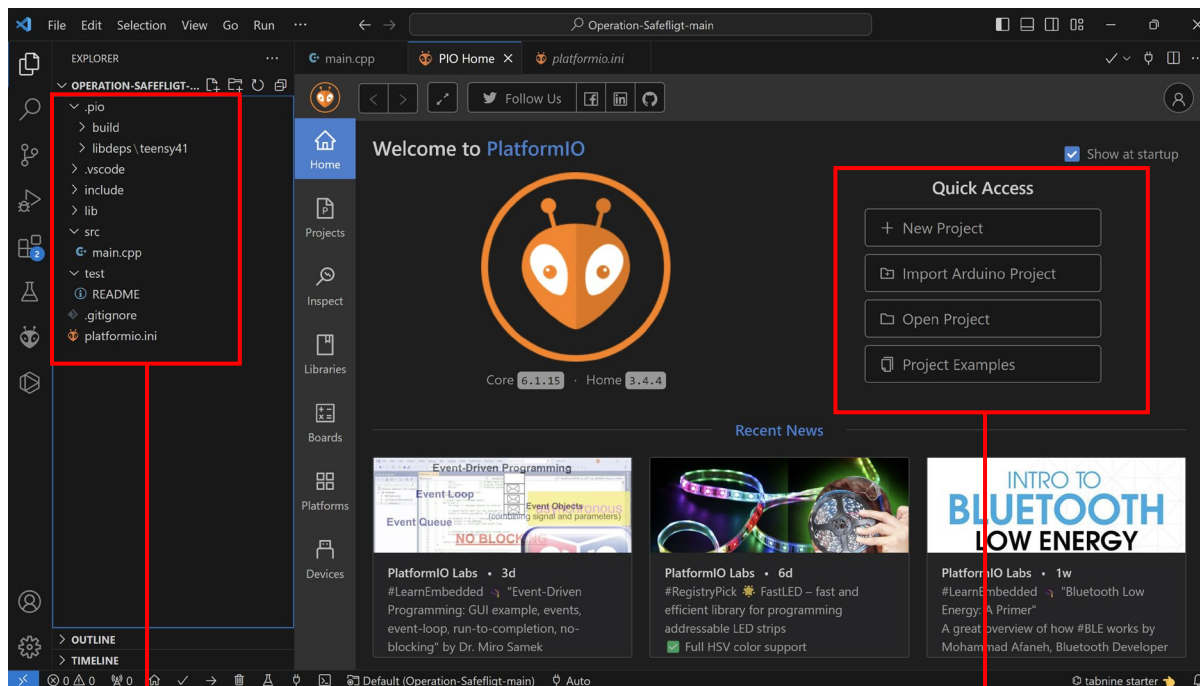


Abbildung 1: Aufbau PlatformIO (Quelle: Screenshot von Visual Studio Code)

Übersicht über das Aktuell Geöffnete Projekt. Die Main Datei befindet sich im src Ordner

Das Hauptmenü von PlatformIO mit welchem man Projekte öffnet, importiert und erstellt.

Programmiert wird dann schlussendlich in einer angepassten Version von C++, mit welcher man sehr effizient Mikrocontroller programmieren kann. Wobei bei der Programmierung immer auf einen sehr spezifischen Aufbau geachtet werden muss, damit der Code in PlatformIO ausgeführt und compiled¹ werden kann. Dieser Aufbau sieht wie folgt aus:

¹ Den Code von einer höheren Programmiersprache (hier C++) in Maschinensprache übersetzen

```
#include <Arduino.h>

// put function declarations here:
tabnine: test | explain | document | ask
int myFunction(int, int);

tabnine: test | explain | document | ask
void setup() {
    // put your setup code here, to run once:
    int result = myFunction(2, 3);
}

tabnine: test | explain | document | ask
void loop() {
    // put your main code here, to run repeatedly:
}

// put function definitions here:
tabnine: test | explain | document | ask
int myFunction(int x, int y) {
    return x + y;
}
```

Abbildung 2: Aufbau einer PlatformIO Datei (Quelle: Screenshot von Visual Studio Code)

Für das Verständnis vonnöten ist, dass der Code, welcher im Setup Teil geschrieben wurde, nur einmal ausgeführt wird, und zwar ganz am Anfang. Der Teil, welcher immer wieder wiederholt wird, ist der Loop Teil; hier wird alles geschrieben, was mehrmals aufgerufen werden soll. Es ist in PlatformIO sehr wichtig zu beachten, dass man die Funktionen zuerst definiert, bevor man sie ganz unten im Code schreibt. Zudem, da PlatformIO nicht die Originale Arduino IDE ist, muss man gewisse Dinge zusätzlich importieren, welche schon ab Werk bei der Arduino IDE vorhanden sind. Das Wesentlichste, ohne welches der Code nicht funktionieren würde, ist die «Arduino.h» Library. Diese beinhaltet alle notwendigen Basisfunktionen der Arduino IDE. Diese Library ist absolut notwendig, damit der Code funktioniert und auch richtig compiled werden kann.

Verwendete Ressourcen

Ultraschallsensor (Modell: Hc-04)

1. Allgemeine Informationen über den Sensor

Tabelle 1: Technische Daten Hc-sr04

Der verwendete Ultraschallsensor Hc-sr04 ist einer der beliebtesten Sensoren, welche zur Messung von Distanzen verwendet werden. Er wird in verschiedensten Elektronik- und Robotik Projekten eingesetzt. Auch ich habe diesen Sensor im Rahmen meiner Maturaarbeit benutzt, um zu ermitteln, ob die Drohne auf Kollisionskurs mit einem Objekt ist oder nicht. Der Sensor basiert auf dem Prinzip der Ultraschall-Echomessung, ähnlich dem System, welches auch von Fledermäusen benutzt wird. Um die Distanz zwischen sich und einem Objekt zu ermitteln, sendet der Sensor Ultraschallwellen mit einer Frequenz von 40 kHz aus, welche dann auf einem Objekt reflektiert, werden und als Echo zum Sensor zurückkehren.

Technische Daten Hc-sr04	
Betriebsspannung :	5V DC
Stromaufnahme:	15mA
Messbereich:	2cm – 400 cm
Messwinkel:	15°
Frequenz:	40kHz
Auflösung/Genauigkeit:	3mm

Mögliche Fehlerquellen des Sensors:

Temperatur und Luftfeuchtigkeit: Die Messungen der Ultraschallsensoren basieren auf der Laufzeitmessung von Schallwellen, welche durch Änderungen in der Luftfeuchtigkeit und -temperatur in ihrer Geschwindigkeit erheblich beeinflusst werden. Die Schallgeschwindigkeit wird stärker durch die Temperatur als durch die Luftfeuchtigkeit beeinflusst, höhere Temperaturen führen dazu dass sich die Ultraschallwellen mit einer höheren Schallgeschwindigkeit bewegen. Die Luftfeuchtigkeit bewirkt einen ähnlichen Effekt welcher jedoch geringer, aber nichtsdestotrotz relevant ist. Durch diese Veränderungen können systematische Messfehler auftreten, wenn die Sensorik nicht auf die Umgebungsbedingungen kalibriert ist. (Quelle: (Pepperl+Fuchs, 2024))

Oberflächenbeschaffenheit des Zielobjekts: Die Ultraschallwellen werden je nach Oberflächenbeschaffenheit und oder des Materials des Zielobjektes unterschiedlich gut reflektiert. Glatte, harte Oberflächen reflektieren Schallwellen nahezu vollständig, was die Genauigkeit der Messung stark beeinflusst. Im Gegensatz dazu führen weiche, poröse oder unebene Oberflächen zu einer Absorbierung oder Streuung der Schallwellen, was zu Schwächeren oder Diffusen(unklar, unscharf, verstreut) Reflexionen führt. (Quelle: (Baumer Passion for Sensors, 2024))

Interferenzen und Vibrationen: Durch externe Ultraschallquellen, wie andere Ultraschallsensoren oder stärkere akustische Störungen im gleichen Frequenzbereich von 40kHz können die Ultraschallsensoren interferiert werden. Dadurch können Überlagerrungen der Schallwellen zu Verfälschungen der gemessenen Zeit (Laufzeit) auftreten, was in ungenauen Messungen resultiert. Zudem können durch die mechanischen Vibrationen, welche auftreten, wenn die Motoren der Drohne drehen, Messungenauigkeiten der Sensoren auftreten. Durch eine ungewollte Bewegung des eigentlichen Sensors, kann die Messung der Zeit gestört werden, da sich die Sensorik nicht mehr am gleichen Ort befindet. Durch die mit den Schwingungen auftretenden akustischen Rauscheffekte, welche die Erkennung der reflektierten Signale erschweren kann, die Messungen weiter verschlechtert werden. (Quelle: (Pepperl+Fuchs, 2024))

2. Aufbau und Funktionsweise

Aufbau:

Der Hc-sr04 ist ein Sensor mit der Eigenschaft Ultraschallwellen zu senden und diese auch wieder zu empfangen. Um an einen Mikrocontroller angeschlossen zu werden, besitzt der Sensor vier Ausgänge / Eingänge (sogenannte Pins), welche alle eine bestimmte Funktion übernehmen. Die Stromversorgung des Moduls wird durch die beiden Pins: Vcc (+5V) und GND gewährleistet. Wobei Vcc das positive Terminal des Moduls und GND das negative Terminal ist. Die Kommunikation mit dem Mikrocontroller wird über die Trig und Echo-Pins hergestellt.

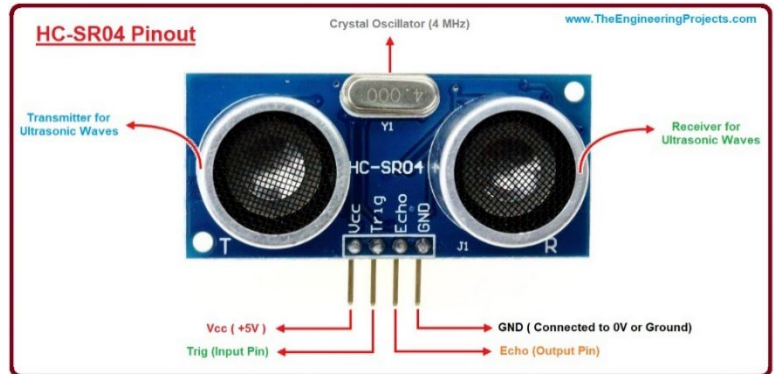


Abbildung 3: Ultraschall Sensor Hc-sr04 Pinout (Quelle: David Watson, <https://www.theengineeringprojects.com/2018/10/introduction-to-hc-sr04-ultrasonic-sensor.html>, abgerufen am 21/07/2024)

Funktionsweise:

Der Transmitter des Sensors entsendet eine Ultraschallwelle mit einer Frequenz von 40kHz (Kilohertz). Dies bedeutet, dass der Sensor 40'000 Zyklen pro Sekunde aussendet, um die Distanz zu einem Objekt zu ermitteln.

Der Ablauf, wie die Ultraschallwellen ausgesendet werden, ist der Folgende:

1. Der Trig Pin erhält vom Mikrocontroller einen 10µs langen elektrischen Impuls, welcher dem Sensor signalisiert, dass er einen Messzyklus starten soll.
2. Nachdem der Sensor das «go» erhalten hat, sendet der Transmitter einen 8-Zyklus Ultraschallstoss aus; diese 8 Ultraschallzyklen besitzen die Frequenz von 40kHz. Die Frequenz von 40kHz wird durch den Oscillator ²(= Oszillator) welcher durch wiederholtes Schwingen die konstante Frequenz erzeugt.



Abbildung 4: Ablauf der Ultraschall-Entsendung (Quelle: How to Mechatronics, <https://howtomechatronics.com/tutorials/arduino/ultrasonic-sensor-hc-sr04/>, abgerufen am 21/07/2024)

3. Nach der Entsendung beginnt die Ausbreitung der Ultraschallwellen mit einer Konstanten Geschwindigkeit (=Schallgeschwindigkeit), bis die Wellen von einem Objekt reflektiert werden

² Ein Oszillator ist eine elektronische Schaltung, die durch Rückkopplung und Verstärkung eine stabile, periodische Schwingung erzeugt, oft mithilfe eines Schwingkreises oder Quarzkristalls.

4. Das Echo der Schallwellen wird dann vom Receiver (=Empfänger), ein Mikrofon-artiges Bauteil, empfangen welches die Schallwellen erkennt.
5. Wenn das Echosignal zurückkehrt, wird die interne Zeitmessung gestartet und durch den Echo-Pin sendet der Sensor ein High Signal aus. Die Zeitmessung endet, wenn das Echosignal vollständig empfangen wurde. Während dieser Zeit bleibt der Echo-Pin auf High. Die Dauer, welche der Echo-Pin auf High ist, wird als Echo-Zeit beschrieben und ist die Grundlage für die Distanzberechnung. Die Formel für die Berechnung ist:

$$\text{Entfernung} = \frac{\text{Echo_Zeit} \times \text{Schallgeschwindigkeit}}{2}$$

3. Programmierung eines HC-sr04 Sensors

Der Hc-sr04 Sensor kann auf verschiedensten Wegen programmiert werden; es ist möglich, den Sensor ohne Libraries zu programmieren, was ich jedoch nicht getan habe. Ich habe aus Effizienzgründen eine Library verwendet, namens newPing³, welche schon in der Arduino IDE integriert wurde und demnach relativ einfach importiert, werden kann (`#include<newPing.h>`). Durch die Verwendung dieser Library war es mir möglich, die Hc-sr04 Sensoren schneller und einfacher auszulesen (siehe später: Vergleich ohne Library zu Library). Jedoch erkannte ich zu spät, dass die newPing Library nicht kompatibel mit meinem Mikrocontroller (Teensy 4.1) ist. Dies aufgrund der Architektur des Mikrocontrollers, welche auf einem ARM-Cortex-Prozessor basiert. (PJRC, 2024). Glücklicherweise wurde die Library schon für einen Teensy 4.1 Prozessor umgeschrieben. Ich habe mich mit dem Autor der «neuen» Library in Kontakt gesetzt und nachgefragt, ob ich ihn in meiner Arbeit zitieren muss oder nicht. Laut ihm sei es in Ordnung, wenn ich ihn nicht erwähne.

Der Link zu seinem GitHub Repository: https://github.com/mjs513/NewPing_t4

Um die abgewandelte newPing Library zu verwenden, muss sie zusätzlich installiert werden, was im Grundsatz gleich funktioniert wie bei jeder anderen Library. Beispielhaft bei der Arduino IDE:

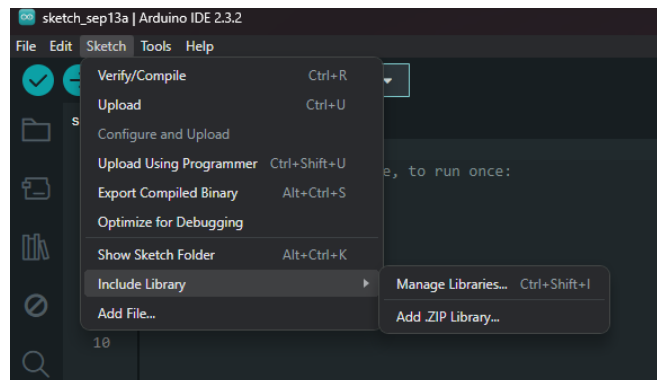


Abbildung 5: Library in der Arduino IDE installieren (Quelle: Cedric Kaufmann, Arduino IDE, erstellt am 13/09/2024)

Wie schon erwähnt kann der Sensor auf verschiedenste Weisen programmiert werden, im Folgenden präsentiere ich Ihnen Ausschnitte, wie der Sensor programmiert wird, mit und ohne Verwendung einer externen Library.

³ <https://www.arduino.cc/reference/en/libraries/newping/>

1. Ohne Externe Library'

```
#define TRIG_PIN 9
#define ECHO_PIN 10

tabnine: test | explain | document | ask
void setup() {
  Serial.begin(9600);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
}

tabnine: test | explain | document | ask
void loop() {
  long duration, distance;

  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  duration = pulseIn(ECHO_PIN, HIGH);

  distance = duration * 0.034 / 2;

  Serial.print("Distanz: ");
  Serial.print(distance);
  Serial.println(" cm");

  delay(500);
}
```

Abbildung 6: Programmierung Hc-sr04 ohne Library
(Quelle: Screenshot von Visual Studio Code)

2. Mit externen Library's (newPing)

```
#include <NewPing.h>

#define TRIG_PIN 9
#define ECHO_PIN 10
#define MAX_DISTANCE 200

tabnine: test | explain | document | ask
NewPing sonar(TRIG_PIN, ECHO_PIN, MAX_DISTANCE);

tabnine: test | explain | document | ask
void setup() {
  Serial.begin(9600);
}

tabnine: test | explain | document | ask
void loop() {
  unsigned int distance = sonar.ping_cm();

  Serial.print("Distanz: ");
  Serial.print(distance);
  Serial.println(" cm");

  delay(500);
}
```

Abbildung 7: Programmierung Hc-sr04 mit NewPing Library
(Quelle: Screenshot von Visual Studio Code)

Erklärung des Codes:

Am Anfang werden die Trig und Echo-Pins definiert (also es wird ihnen ein Pin des Mikrocontrollers zugeschrieben).

Im Setup wird dann eine serielle Kommunikation geöffnet (ermöglicht es Daten auf dem Computer zu sehen) und die oben definierten Pins werden entweder als Output oder Input festgelegt (Trig = Output, Echo = Input)

Im Loop wird dann die Messung ausgeführt (siehe roter Kasten). Indem der Trig Pin aktiviert und deaktiviert wird, => es werden die Ultraschallwellen ausgesendet.

Basieren auf der Zeit zwischen dem Entsenden der Ultraschallwellen und dem Empfangen der reflektierten Wellen, durch den Echo Pin wird dann mit der Formel: $\text{Zeit} * 0.034 / 2$ die Distanz berechnet. 0.034 kommt daher, dass der Schall 343 M/s zurücklegt. Durch 2 wird dividiert, da die Wellen den Weg zweimal zurücklegen.

Erklärung des Codes:

Zuerst wird die Library importiert (#include <NewPing.h>).

Dann, wie auch schon beim obigen Code, werden alle Variablen definiert, zusätzlich muss noch eine maximale Distanz festgelegt werden.

Initialisiert wird der Sensor dann als eine NewPing Klasse mit dem Namen Sonar. Um den Sensor richtig zu initialisieren, müssen die Attribute angegeben werden. Dadurch weiß der Code, welcher Trig und Echo Pin er verwenden soll und die maximale Messdistanz.

Die eigentliche Messung findet dann im Loop-Teil statt (siehe roter Kasten).

Der Rest des Codes ist nur für die Übersicht und die Auslesung der Daten zuständig.

Time Of Flight Sensor (VL53L0X)

1. Allgemeine Informationen über den Sensor

Tabelle 2: Technische Daten VL53L0x Sensor

Wie auch schon der Hc-Sr04 Sensor (siehe Seite: 8) kann man mit diesem Sensor Distanzen zwischen zwei Punkten, dem Sensor und einem anderen Objekt (Bsp. Eine Wand, Tisch, etc.), messen.

Der Unterschied zum Hc-sr04 Sensor besteht darin, dass der VL53L0X (ein Time-of-Flight Sensor, kurz ToF)

Technische Daten VL53L0x	
Betriebsspannung :	3.3V – 5V
Stromaufnahme:	20mA
Messbereich:	0cm – 200 cm
Wellenlänge	940nm
Auflösung/Genauigkeit:	±3%

nicht auf Ultraschalltechnik basiert, sondern auf Messungen mittels Laser. Dies führt dazu, dass diese Sensoren nicht empfindlich auf Schwingungen in der Luft sind, welche zum Beispiel von den Motoren (genauer durch die Drehung der Propeller) produziert werden. Da Licht nicht empfindlich auf Temperatur und Feuchtigkeit ist, so wie es die Schallgeschwindigkeit ist (Digikey, 2024), werden die Messungen immer mit der gleichen Geschwindigkeit ($v \approx 300'000 \text{ km/s}$) durchgeführt. Zu beachten, dies ist nur die Geschwindigkeit der ausgesendeten Lichtstrahlen, die Distanz wird später durch die Zeit berechnet.

Mögliche Fehlerquellen des VL53L0x Sensors

Umgebungslicht: Externe Infrarotquellen, welche Infrarotstrahlung im Wellenlängenbereich von 940 nm absondern, können das Signal-Rausch-Verhältnis ⁴(SNR) des VL53L0x Sensors signifikant stören und beeinträchtigen. Die Fotodiode des VL53L0x Sensors kann durch zu viel Infrarotstrahlung übersättigt werden, was dazu führt, dass die Diode eine verringerte Empfindlichkeit hat, eine verzerrte Signalverarbeitung und schlimmstenfalls potenzielle Messfehler aufzeichnet. Um dies zu kompensieren, implementiert der VL53L0X einen adaptiven Algorithmus zur Hintergrundlichtunterdrückung, der die Integrationszeit und Verstärkung dynamisch anpasst.

Mehrfachreflexionen: In Umgebungen mit komplexen Strukturen können Mehrfachreflexionen zu einer Mehrfachdeutung der Laufzeitmessungen führen. Der VL53L0x Sensor verwendet darum einen Algorithmus, welcher mehrere Ziele anvisieren kann und somit bis zu zwei Reflexionen pro Messzyklus identifizieren kann, was die Wahrscheinlichkeit auf eine Fehlmessung verringert.

Staub und Schmutz: Sammeln sich Staub und Schmutz auf der Oberfläche des Sensors an, kann dies zu einer undurchdringlichen Schicht für den Laser führen. Dies hat zur Folge, dass die Genauigkeit der Messung beeinträchtigt wird, durch die Beeinträchtigung des Laserstrahls.

Quellen: (Wikipedia, 2024), (Wikipedia, 2024), (Tang, 2021), (Schultheiss, 2024)

⁴ Signal-Rausch-Verhältnis ist ein Mass für die technische Qualität eines Nutzungssignals, hier die Qualität der Empfangen Laser

Funktionsweise Aufbau:

Der VL53L0X Sensor hat einen eingebauten Laseremitter, welcher ihm ermöglicht, Laserstrahlen zu entsenden; durch den eingebauten Empfänger wird es ihm auch ermöglicht, die ausgesendeten Signale wieder auszulesen. Diese beiden Elemente sind im rot markierten Kasten vorzufinden. Da die beiden Komponenten so klein sind, sind sie von Auge nicht mehr zu erkennen. Der Sensor besitzt von vier bis sechs Ausgänge / Eingänge für Signale je nach Modell, wobei die letzten zwei Ausgänge / Eingänge optional sind und nicht wirklich gebraucht werden. In diesem Beispiel haben wir einen Sensor mit sechs Aus- oder Eingängen, namentlich:

VIN, GND, SCL, SDA, GPIO, XSHUT. Die beiden Pins VIN und GND sind wie auch schon beim Hc-sr04 Sensor für die Stromversorgung zuständig, wobei der VL53L0X auf 5 Volt Logik basiert, welche er über den VIN Pin aufnimmt. Die Kommunikation mit dem Mikrocontroller erfolgt über ein Protokoll namens I2C⁵, welches über die SCL und SDA Pins abläuft. Da die letzten Pins optional sind, werde ich nicht weiter auf diese eingehen.



Abbildung 8: VL53L0X Sensor (Quelle: Botland, <https://botland.de/eingestellte-produkte/10315-vl53l0x-flugzeit-i2c-abstandssensor-adafruit-3317-5904422315481.html>, abgerufen am 31/08/24)

Funktionsweise:

Der VL53L0X Sensor funktioniert wie oben erklärt auf dem Time-of-Flight System, welches mittels Laseremission die Distanz misst. Der Ablauf sieht wie Folgt aus:

1. Der Sensor entsendet einen unsichtbaren Laserstrahl (welcher sich im Infrarot Bereich des Lichtspektrums befindet) mittels der auf ihm verbauten Sensorik

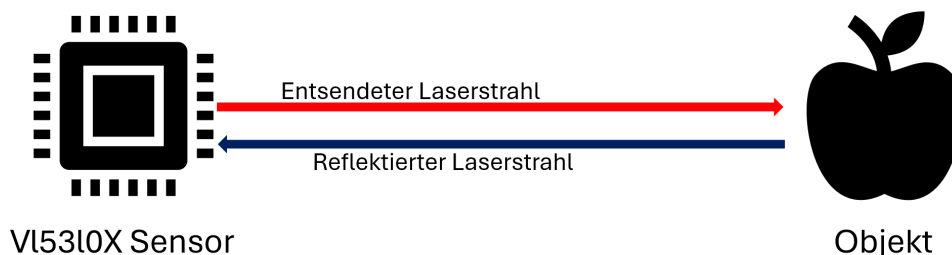


Abbildung 9: Funktionsweise eines VL53L0X Sensors (Quelle: Piktogramm aus Word, Cedric Kaufmann, erstellt am 01/09/2024)

2. Die entsendeten Laseremissionen werden dann auf Objekten reflektiert (Siehe Abbildung 9), die reflektierten Laseremissionen werden wieder vom VL53L0X Sensor wahrgenommen. In einem letzten Schritt werden die berechneten Distanzen per I2C-Kommunikation an den Mikrocontroller gesendet, unter der Bedingung, dass dieser danach gefragt hat.

⁵ Ein serielles Kommunikationsprotokoll, das in der Elektronik verwendet wird, um Daten zwischen verschiedenen Komponenten wie Sensoren und Mikrocontrollern auszutauschen.

2. Programmierung eines VL53L0X Sensors

Ein VL53L0X kann aufgrund der Komplexität nur mit einer Library programmiert werden. Es gilt zu beachten, dass es im Grundsatz möglich ist, den Sensor auch ohne externe Library zu programmieren, jedoch benötigt dieser Ansatz ein tiefes Verständnis des Sensors und des I2C-Kommunikationsprotokolls. Welches ohne die Hilfe von Libraries eher schwierig ist zu programmieren, da sie direkte Anbindung an die Schnittstellen des Mikrocontrollers und des Sensors benötigt, was sehr umständlich zu programmieren ist. Deshalb habe ich in meinem Code eine Library verwendet. Nämlich die VL53L0X Library ⁶des Adafruit Teams (Adafruit, 2024).

```
#include <Wire.h>
#include "Adafruit_VL53L0X.h"

// Erstelle ein Objekt für den Sensor
Adafruit_VL53L0X lox = Adafruit_VL53L0X();

void setup() {
  Serial.begin(115200);
  // Initialisiere den I2C Bus
  Wire.begin();

  // Initialisiere den Sensor
  if (!lox.begin()) {
    Serial.println(F("Konnte VL53L0X nicht finden. Stelle sicher, dass der Sensor richtig angeschlossen ist!"));
    while (1);
  }
  Serial.println(F("VL53L0X gestartet!"));
}

void loop() {
  // Struktur zum Speichern der Messdaten
  VL53L0X_RangingMeasurementData_t measure;

  // Starte eine Messung
  lox.rangingTest(&measure, false); // Der zweite Parameter ist 'true' für eine genauere Messung (langsamer)

  // Prüfe, ob die Messung gültig ist
  if (measure.RangeStatus != 4) { // Status 4 bedeutet "out of range"
    Serial.print(F("Entfernung (mm): "));
    Serial.println(measure.RangeMilliMeter);
  } else {
    Serial.println(F("Außerhalb des Messbereichs"));
  }

  delay(100); // Warte 100ms zwischen den Messungen
}
```

Abbildung 10: Programmierung eines VL53L0X Sensors (Quelle: Codeausschnitt von Visual Studio Code, Cedric Kaufmann, erstellt am 01/09/2024)

Erklärung des Codes:

Am Anfang des Codes müssen die nötigen Libraries importiert werden, die «Wire.h» library ist eine Standardbibliothek der Arduino IDE, sie ist die Bibliothek, welche die I2C-Kommunikation ermöglicht, ohne dass man selbst noch zusätzlichen Code dafür schreiben muss. Die «Adafruit_VL53L0X» library ermöglicht die Steuerung des Sensors. Um die I2C Kommunikation zu starten, muss der Code Teil «Wire.begin()» im Setup Teil des Codes geschrieben werden. Im Orangen Kasten wird dann der VL53L0X Sensor initialisiert und getestet, ob dieser auch funktioniert (Die Instanz des Sensors wurde im gelben Kasten erstellt).

⁶ https://github.com/adafruit/Adafruit_VL53L0X

Die eigentliche Messung findet dann im Loop-Teil des Codes statt. Dort wird zuerst eine Variable erstellt, welche es ermöglicht die gemessenen Daten zu speichern. Die Messung wird dann im Blauen Kasten durchgeführt. Im restlichen Teil des Codes wird nur noch verifiziert ob die Messung valide ist oder nicht, diesen Schritt musste ich aus Effizienzgründen in meiner Arbeit auslassen.

Multiplexer (TCA9548A)

1. Allgemeine Informationen über das Modul

Der Multiplexer (TCA9548A) wurde entwickelt um bis zu 8 separate I2C Geräte (sog. Busse) über einen einzelnen gemeinsamen I2C-Bus zu verwenden. Durch die Verwendung eines Multiplexers wird es ermöglicht das man auf einem Bus acht Geräte anstatt einem verwenden kann. Dies funktioniert darum, da der Multiplexer jedem angeschlossenen Gerät einen anderen Bus (oder Kanal) zuordnet, was eine Verwechslung ausschliesst.



2. Funktionsweise / Verwendungszweck

Der Multiplexer (TCA9548A) dient dazu, mehrere I2C-Geräte mit gleichen Adressen (also mehrere gleiche Geräte, meistens) in einem einzigen System zu betreiben. Dies ist hilfreich, wenn mehrere Sensoren, Displays oder andere Geräte mit identischen I2C-Adressen verwendet werden, was ansonsten zu Konflikten führen würden.

Abbildung 11: Multiplexer (TCA9548A)
(Quelle: Baastelgarage.ch, <https://www.baastelgarage.ch/8-channel-i2c-multiplexer-tca9548a>, abgerufen am 13/09/2024)

Technische Daten	
Betriebsspannung:	1,65V – 5,5V
Kommunikationsprotokoll:	I2C (Inter-Integrated Circuit)
Anzahl Kanäle:	8
Einstellbare I2C-Adresse:	0x70 bis 0x77
Bidirektionale Kommunikation:	Beides Master und Slave

Tabelle 3: Technische Daten Multiplexer

Funktionsweise:

- Der Multiplexer (TCA9548A) teilt die I2C-Busse in 8 Verschiedene Kanäle auf
- Jeder einzelne dieser Kanäle ist separat ansteuerbar, durch das I2C-Protokoll kann festgelegt werden welcher Kanal (Bus) gerade angesteuert werden soll.
- Der Master (Bsp. Ein Mikrocontroller) kommuniziert über den normalen I2C-Bus, also über die SDA und SCL Leitungen, welcher er besitzt, der Multiplexer auf der anderen Seite leitet den Verkehr zu einem der 8 möglichen Kanäle um.
- Durch Schreiben eines speziellen Steuerbefehls (einer 8-Bit-Nummer, wobei jedes Bit für einen der 8 Kanäle steht) entscheidet der Master, welcher Kanal geöffnet werden soll.

Verwendungszweck:

- Der Multiplexer (TCA9548A) wird verwendet, um Adressen Konflikte zwischen I2C-Geräten in einem System zu vermeiden.
- Er ermöglicht mehrere gleiche Geräte über einen einzigen I2C-Bus zu betreiben

Quellen: (Random Nerd Tutorial, 2024), (Ewald, 2024)

Mikrocontroller (Teensy 4.1)

1. Allgemeine Informationen über den Mikrocontroller

Der Teensy 4.1 Mikrocontroller wurde entwickelt von PJRC. Es ist ein leistungsstarker und flexibler Mikrocontroller, welcher seine Anwendung in Applikationen findet, welche eine hohe Performance (= viel Rechenleistung) erfordern. Trotz seiner geringfügigen Grösse ist der Teensy 4.1 mit starker Hardware ausgestattet, die es ihm ermöglicht auch Anspruchsvolle Programme ohne Probleme auszuführen.

Der Teensy 4.1 ist ideal für Projekte geeignet, welche viel Rechenleistung, viele Anschlüsse für Sensoren (= viel I/O Kompatibilität besitzt, siehe Abbildung 12) und externe Features wie Ethernet und SD Karten support benötigen.

Überblick über die Eigenschaften des Teensy 4.1

Wichtigste Eigenschaften:

- Prozessor: ARM Cortex-M7 welcher mit 600 MHz⁷ läuft.
- Interner Speicher:
 - o 1024K Ram
 - o 8Mb Flash (Speicherort für die Programme)
- USB interface: Stromversorgung, High-speed Kommunikation mit einem seriellen Gerät, Schnittstelle für andere serielle Geräte in Projekten.
- Ethernet: 10/100 Mbit
- SD Karten Support
- Betriebsspannung: 3.3V

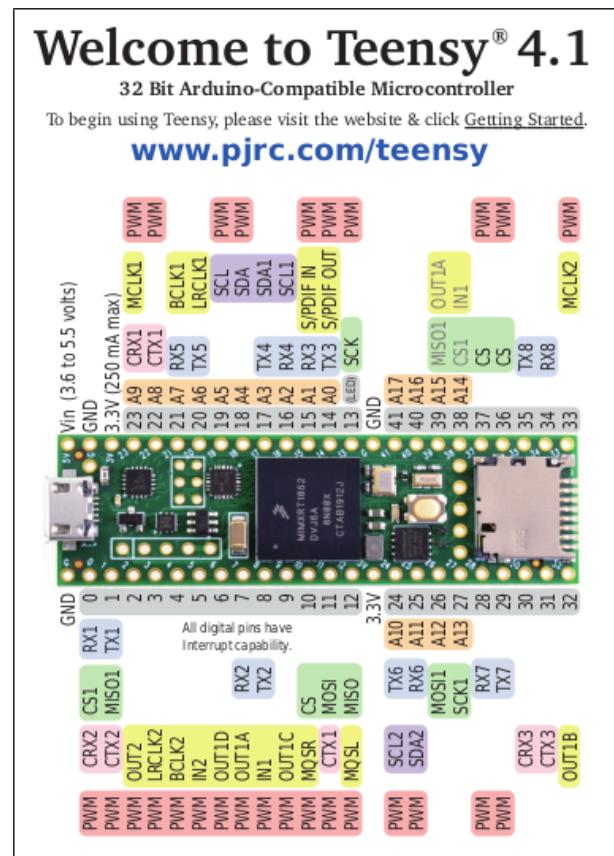


Abbildung 12: Pinout Teensy 4.1 (Quelle: PJRC, <https://www.pjrc.com/store/teensy41.html>, abgerufen am 17/09/24)

⁷ MHz ist die Abkürzung für Megahertz und bedeutet das ein Prozessor eine Million Rechenzyklen pro Sekunde ausführen kann.

Input/Output Fähigkeiten:

- Digitale I/O Pins: 55 gesamt
- Analoge Eingänge: 18
- PWM Ausgänge: 18
- Serielle Ports: 8
- SPI⁸: 3
- I2C: 3

Zusätzliche Features:

- Battery-backed RTC ⁹(batteriegepufferte Echtzeituhr): Ist eine Uhr, welche auch weiterläuft, wenn der Mikrocontroller vom Strom getrennt wird, dies wird ermöglicht durch eine interne Batterie.
- Interner Temperatur Sensor
- Hardware Basierter Random Number Generator

Quelle: (PJRC, 2024)

2. Verwendung

Der Teensy 4.1 ist der zweite Rechner in der Drohne, er ist zuständig für das Abrufen der Sensormessungen, die Berechnungen für das Ausweichmanöver und das Empfangen / Weiterleiten der PPM Signale. Dies geschieht durch ein von mir selbst geschriebenes Skript, welches auf dem Teensy 4.1 läuft. Es sorgt dafür das in einem ersten Schritt geschaut wird in welchem Modus der Teensy 4.1 agieren soll. Programmiert sind 3 Modi: Einer bei welchem das ganze Kollisionspräventionssystem ausgeschaltet ist und der Pilot volle Kontrolle über die Drohne besitzt. Ein zweiter bei welchem die Drohne den Piloten mit der Höhenkontrolle unterstützt und verhindern soll, dass die Drohne nicht in den Boden oder die Decke fliegt. Im letzten Modus unterstützt die Drohne den Piloten in alle Bewegungsachsen (X;Y;Z) und verhindert, wenn möglich eine Kollision (kann aufgrund verschiedensten Faktoren wie: Geschwindigkeit, Wind, nicht erkannte Objekte, nicht funktionieren). Um dies zu erreichen, liest der Teensy 4.1 so schnell wie möglich die Sensordaten aus und evaluiert sie mit einem Algorithmus, um mögliche Kollisionen zu erkennen. Falls Kollisionen wahrscheinlich sind, berechnet der Teensy 4.1, wie weit und wohin die Drohne ausweichen muss. Es ist wichtig zu erwähnen, dass die Drohne an keiner Stelle des Programmes die Kontrolle selbst übernimmt. Um eine Kollision zu vermeiden, ändert die Drohne lediglich die einkommenden PPM Signale. Erkennt der Teensy 4.1 beispielsweise das sich vor der Drohne ein Objekt befindet wird er das Signal welches zuständig ist für die Bewegung nach vorne um einen im Code bestimmten Faktor in die Gegenrichtung des Objektes manipulieren.

⁸ SPI ist die Abkürzung für Serial Peripheral Interface und ist ein Protokoll welches schnelle Datenübertragung zwischen Mikrocontroller und einem oder mehreren Peripheriegeräten ermöglicht.

⁹ RTC: Real-Time Clock (Echtzeituhr)

Kommunikationsprotokolle

1. Funkprotokoll PPM

Um das Funkprotokoll besser zu verstehen, muss man zuerst einmal wissen, was es genau ist. PPM oder auch Puls-Position-Modulation ist eine Methode, mit welcher man durch die Manipulation der Stromlänge, also wie lange ein elektrisches Modul Strom bekommt, regulieren kann, wie dieses Modul reagieren wird. Um dies besser zu verstehen, schauen wir zuerst den «kleinen» Bruder der Puls-Position-Modulation an, die sogenannten PWM-Signale. Diese sind grundsätzlich die Bausteine eines PPM-Signals, da mehrere PWM-Signale in einem PPM-Signal zusammengesetzt sind. (GeeksforGeeks, 2024)

2. PWM Signale

Funktionsweise:

Die pulse-width modulation (PWM) ist eine grundlegende Technik zur Steuerung der Energiemenge, die an ein elektrisches Gerät geliefert wird. PWM wird als der «kleine Bruder» der pulse-position modulation (PPM) bezeichnet. Durch das schnelle Ein- und Ausschalten des Signals kann die Leistung des Moduls variiert werden. Die Frequenz des Signales bleibt bei PWM-Signalen konstant; was sich ändert, ist das Verhältnis zwischen EIN-Zeit (ON-Time) und AUS-Zeit (OFF-Time). Dieses Verhältnis bestimmt, wie die Energiemenge während einer Periode verteilt wird und ermöglicht eine präzise Steuerung der Energiezufuhr eines Moduls. Das Verhältnis zwischen EIN- und AUS-Zeit wird als Tastverhältnis oder Duty Cycle bezeichnet.

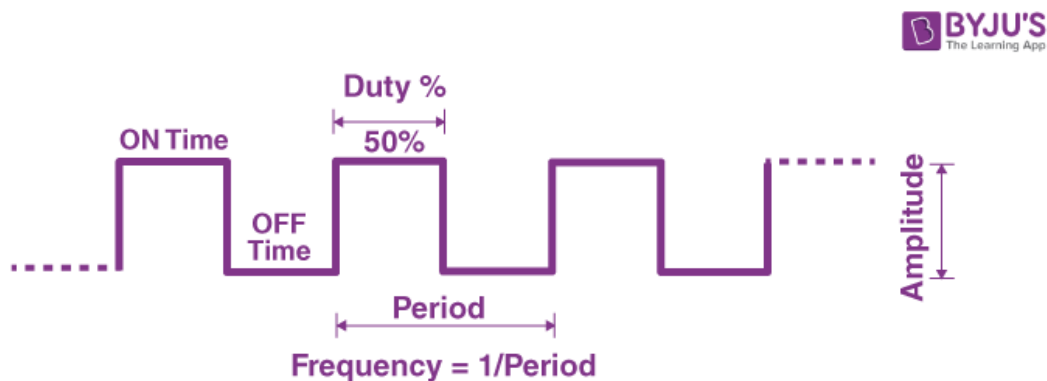


Abbildung 13: Aufbau eines PWM-Signal (Quelle: BYJU'S, <https://byjus.com/physics/pulse-width-modulation/>, abgerufen am 08/08/2024)

Ein PWM-Signal setzt sich aus einer Serie von Perioden, welche jeweils eine Ein- und Auszeit besitzen (= On Time und Off Time). Die Periode eines PWM-Signals startet mit dem Beginn der Ein-Zeit und endet mit der Aus-Zeit. Die Ausgangsleistung eines Moduls wird dann durch die Variation der Länge der Ein-Zeit gesteuert. Denn innerhalb einer festen Periode lässt sich durch längere Ein-Zeit die Energiemenge, welche ein Modul erhält, steuern, da sich dann das Verhältnis zwischen Ein- und Auszeit ändert. Dies hat zur Folge, dass je länger die Ein-Zeit im Vergleich zur Gesamtdauer der Periode ist, desto höher ist die an das Gerät gelieferte Energiemenge.

Wichtige Parameter eines PWM-Signals:

- **EIN-Zeit:** Dauer, während der das Signal hoch ist => Das Modul wird mit Strom versorgt. Durch die Variation der Ein-Zeit kann die Energiemenge, welche das Modul erhält, gesteuert werden.
- **AUS-Zeit:** Dauer, während der das Signal niedrig ist => Die Zeit, in welcher das Modul keinen Strom erhält und somit nichts macht.
- **Periode:** Die Gesamtdauer eines Zyklus des Signals, von Start (EIN-Zeit) zum Ende (AUS-Zeit).
- **Frequenz:** Die Anzahl der Perioden pro Sekunde, also wie viel Mal das Signal pro Sekunde wiederholt wird.
- **Tastverhältnis (Duty Cycle):** Der Prozentsatz der Periode, bei welcher der das Signal EIN ist ($\frac{EIN-Zeit}{Periode} * 100$)

Anwendungsbeispiel von PWM-Signalen:

Um die Funktionsweise von PWM-Signalen besser zu verstehen, widmen wir uns in einem Beispiel einem Modul, welches oft via PWM-Signal gesteuert wird. Die Rede ist von einem DC-Motor, welcher in unserem Beispiel mithilfe eines PWM-Signals gesteuert wird. Die Geschwindigkeit, mit welcher sich der Motor drehen kann, hängt direkt zusammen mit der Menge an Strom, welcher er bekommt. Durch das PWM-Signal wird eben jene Strommenge gesteuert, was uns ermöglicht, die Geschwindigkeit von DC-Motoren ziemlich genau zu steuern. Durch das PWM Signal wird die mittlere Spannung des Motors angepasst. Ein höheres Tastverhältnis führt zu einer höher mittleren Spannung und damit zu einer höheren Motordrehzahl. Die PWM Signaltechnik ist sehr vielseitig und wird in verschiedensten Bereichen eingesetzt, darunter wie oben erklärt die Steuerung von Motoren oder die Helligkeitsregulierung von LEDs. Sie bietet eine sehr effiziente Methode, die elektrische Leistung durch die bereitgestellte Energiemenge zu steuern.

3. PPM Signale

Ein PPM-Signal besteht im weitesten Sinne aus 6–8 (im Normalfall) zusammengehängten PWM-Signalen. Genauer gesagt: In einem PPM-Signal werden mehrere Informationskanäle zu einem einzelnen Signal kombiniert, wobei die relative Position des Pulses die Darstellung eines Kanals darstellt. Was PPM-Signale von PWM-Signalen unterscheidet, ist, dass bei einem PPM-Signal die Position eines Pulses innerhalb eines festgelegten Zeitrahmens variiert wird, um Informationen zu übermitteln. Im Gegensatz zur Pulsweitenmodulation (PWM), bei welcher die Breite eines Signals variiert wird, also durch die Variation der On und Off Time, ist bei der PPM die exakte Position des Pulses entscheidend. Das bedeutet, dass die Information nicht durch die Dauer des Signals selbst (wie bei PWM), sondern durch den Zeitpunkt innerhalb einer festen Periode übertragen wird.

PPM-Signale arbeiten ähnlich wie PWM-Signale, jedoch wird bei PPM-Signalen die Position eines Pulses innerhalb einer festen Zeitperiode verwendet. Das heisst, ein PPM-Signal besteht aus einer Reihe verschiedenster Pulse, welche durch den Zeitabstand zwischen den Pulsen definiert wird (= die Position des Pulses, daher der Name: Pulse Position Modulation), die eigentliche Information des Signales ist durch den Zeitabstand codiert. Die Frequenz eines PPM-Signals kann stark von Anwendung zu Anwendung variieren, jedoch hat ein typisches PPM-Signal eine Frequenz von 50Hz, das heisst so viel wie, dass alle 20 Millisekunden ein vollständiger Satz an Kanälen gesendet wird.

Aufbau eines PPM-Signals:

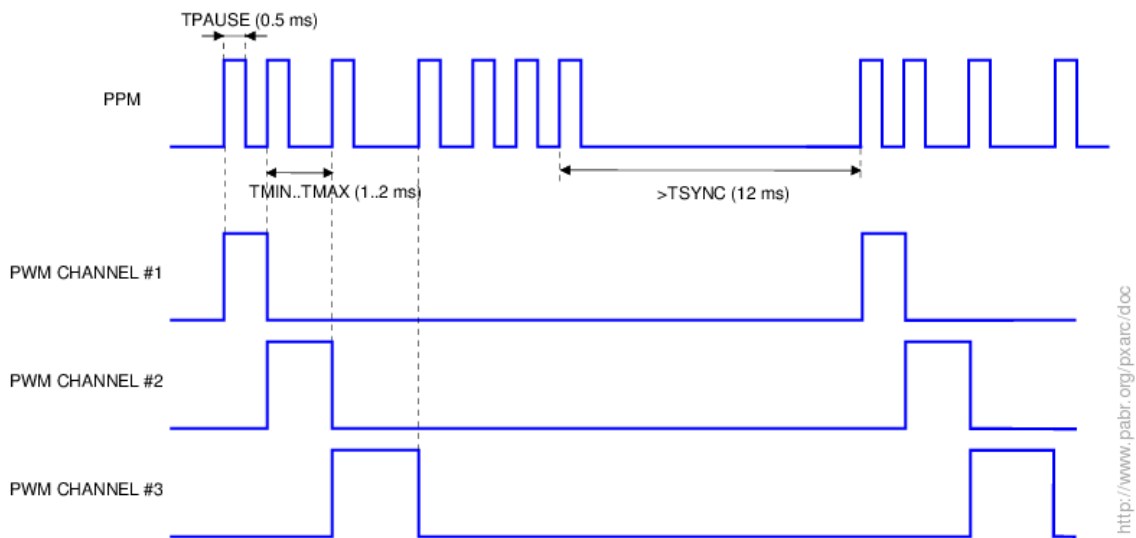


Abbildung 14: PPM-Signal (Quelle: Arduino Forum, <https://forum.arduino.cc/t/generate-ppm-pulses-for-radio-control/616442>, abgerufen am 21/09/24)

Wie in Abbildung 14 zu erkennen, kann ein PPM-Signal in verschiedene Kanäle aufgespalten werden. Zum Start des Signals wird ein Startpuls gegeben, dann folgt Kanal Eins mit seinem Kanalpuls, welcher dann für eine gewisse Zeit auf High schaltet. Die Länge zwischen Kanalpuls Eins und Zwei bestimmt die codierte Kanalinformation von Kanal Eins. Am Ende des letzten Kanals wird ein Syncpuls angehängt, um die Periodendauer noch aufzufüllen, sodass das ganze Signal nicht aus der Synchronisierung fällt und Fehler auftreten.

Anwendungen:

PPM-Signale finden häufige Verwendung in der RC-Welt, um verschiedenste Geräte wie Drohnen, Helikopter oder Flugzeuge zu steuern. Da durch die Aufteilung in verschiedene Kanäle alle wichtigen Informationen für die Steuerung mit nur einem Signal übermittelt werden können.

Am Beispiel einer Drohne könnte Kanal Eins den Throttle (Schub) darstellen und die weiteren Kanäle die Informationen für Pitch, Yaw und Roll der Drohne speichern. Auch in meinem Projekt verwende ich das Funkprotokoll PPM, was es einfach macht, die verschiedensten Signale kompakt und schnell zu versenden.

Entwicklung der 3d gedruckten Teile

Um die verschiedenen Komponenten der Drohne zu entwickeln, welche ich zusätzlich zur Grundausstattung der Drohne hinzugefügt habe, musste ich einen Weg finden, um diese möglichst platzs- und gewichtssparend an der Drohne anzubringen. Die Wahl der Methode fiel ziemlich schnell auf 3D Druck, da dieser eine kostengünstige Alternative zu z. B. CNC-gefrästen Teilen oder auch Spritzguss darstellt, welche mir zuerst als mögliche Methoden in den Sinn kamen. Aus Kostengründen entschied ich mich jedoch dazu, dem 3D Druck eine Chance zu geben, zumal ich schon mehrere 3D Drucker besass und ich mir die nötigen Fähigkeiten bereits angeeignet hatte, um die Teile zu entwerfen und zu zeichnen.

1. Erläuterung der verwendeten Software

Die verschiedenen Komponenten der Drohne, welche ich zusätzlich gefügt habe, wurden mit einer CAD-Software ¹⁰ namens Fusion 360 von Autodesk entwickelt. Für den Privatgebrauch wird eine limitierte Version von Fusion 360 gratis zur Verfügung gestellt, solange man mit der Entwicklung der Teile kein Geld verdient. Mit einer CAD-Software wird es einem ermöglicht, genaue technische Konstruktionen in einem Computer zu zeichnen und diese später in verschiedensten Dateiformaten zu exportieren, für meine Arbeit habe ich das Format. stl verwendet, welches sehr gut geeignet ist für den 3D-Druck.

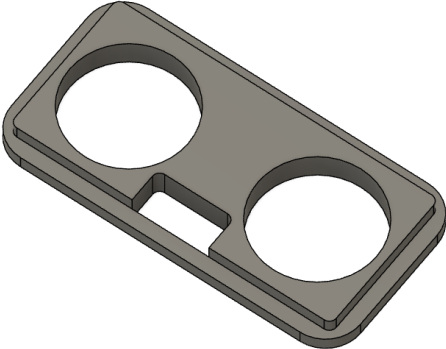
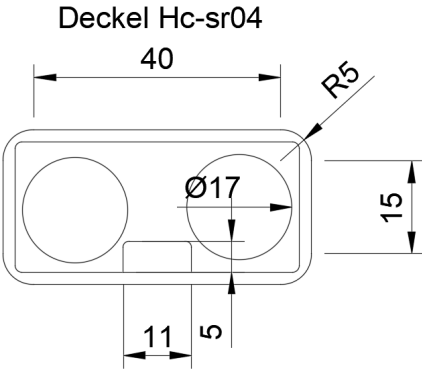
2. Überblick über die entwickelten Bauteile

Die Bauteile wurden in mehreren Schritten entwickelt, mit verschiedensten Iterationen und Versionen der verschiedenen Einzelteile. Durch das CAD-Programm war dies relativ einfach, da dieses Programm die Bauteile, welche gerade entwickelt werden, schon dreidimensional darstellt und so einen guten Überblick verschafft. Der Entwicklungsprozess dauerte je nach Bauteil verschieden lange. Im Ganzen habe ich sieben einzelne Bauteile, welche ich im Folgenden erklären werde.

¹⁰ CAD steht für "Computer-Aided Design" und bezeichnet Software, die zur Erstellung präziser Zeichnungen und Modelle verwendet wird.

Relevante Teile für die Hc-sr04 Sensorik:

1. Gehäuse für die Sensorik

Deckel	
 Abbildung 15: 3D Modell des Deckels für die Hc-sr04 Sensoren (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	Der Deckel des Gehäuses für die Hc-sr04 Sensoren wurde entwickelt, um das Gehäuse bündig zu schliessen. Deshalb hat der Deckel auch die zwei runden Aussparungen für den Receiver und Transmitter des Hc-sr04 Sensors. Das Loch zwischen den beiden Aussparungen (den Kreisen) ist eine Aussparung für den Oszillator, welcher beim Hc-sr04 eine rechteckige Form besitzt. Um den Deckel gut mit dem Gehäuse zu verbinden, integrierte ich den Rand um die zentrale Einheit, welcher dann bündig auf dem Rand des Gehäuses aufliegt. Der Deckel wird durch die Reibung, welche entsteht, wenn er in das Gehäuse gedrückt wird, gehalten. Deshalb müssen die Ränder des Deckels und des Gehäuses möglichst abgerundet sein, da dadurch mehr Stabilität garantiert werden kann. Dadurch wird die Kontaktfläche zwischen den beiden Teilen grösser und der Druck wird besser verteilt. Dies sorgt für eine stabilere Passform und reduziert das Risiko von Beschädigungen oder Lockerungen, da die Reibung besser verteilt wird und die Teile sich weniger leicht verschieben oder lösen. Das untere Bild zeigt die technischen Abmessungen des Deckels (ohne die Höhe).
 Abbildung 16: Technische Zeichnung des Deckels für die Hc-sr04 Sensoren (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	

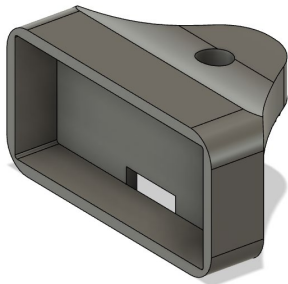
Gehäuse

Abbildung 17: 3D Modell der Vorderseite des Hc-sr04 Gehäuses
(Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)

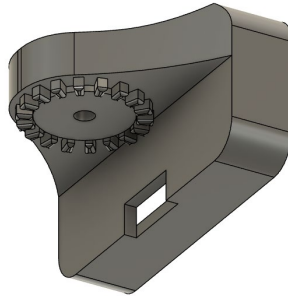


Abbildung 18: 3D Modell der Rückseite des Hc-sr04 Gehäuses
(Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)

Die Hc-sr04 Sensoren brauchen ein eigenes Gehäuse, um sie vor potentiellen Schäden durch Unfälle oder etwaige

Ereignisse zu schützen. Um dies zu erreichen, wurden die Wände des Gehäuses extra dick gemacht. Die starken Rundungen der Ecken waren essenziell für die Stabilität da: Dadurch die Kontaktfläche zwischen den beiden Teilen vergrößert wird und der Druck besser verteilt wird. Dies sorgt für eine stabilere Passform (mit dem Deckel) und reduziert das Risiko von Beschädigungen oder Lockerungen, da die Reibung besser verteilt wird und die Teile sich weniger leicht verschieben oder lösen lassen. Um die Kabel der Sensorik sicher und effizient aus dem Gehäuse herauszubringen, entwarf ich am unteren Ende des Gehäuses eine Aussparung, aus welcher die Pins / Anschlüsse der Sensorik herausragen und dadurch eine geeignete Anschlussstelle entstehen kann.

Das Gehäuse wird durch die zahnradartige Struktur (siehe Abbildung 18) mit dem passenden Gegenstück auf dem Verlängerungsarm montiert.

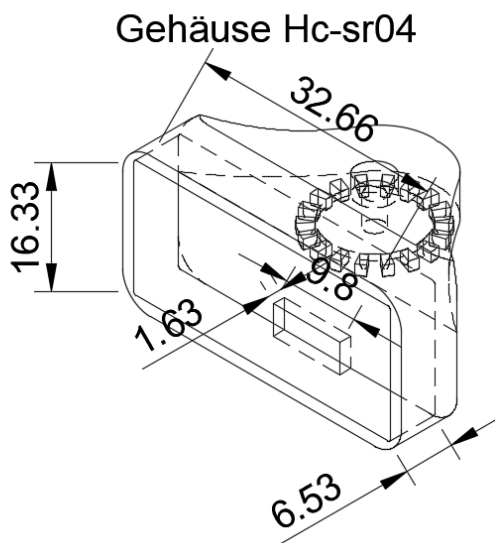


Abbildung 19: Technische Zeichnung des Gehäuses für die Hc-sr04 Sensoren
(Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)

2. Befestigung am Drohnen Arm:

Die Verbindung zwischen dem Arm der Drohne und den Hc-sr04 Sensoren erfolgt über zwei zwischen Teile, welche ich später erklären werde. Dies war mit Abstand der komplizierteste Teil der Entwicklung der 3D gedruckten Teile. Da durch die Gegebenheiten der Drohne der Entwicklungsprozess noch erschwert wurde, zum Beispiel: Die Motoren haben Schrauben, welche durch den Arm der Drohne gehen, um diese Schrauben herum musste ich die Teile entwickeln. Ein zweites Problem war auch die Befestigung des Verbindungsstückes zwischen dem Teil, welches auf dem Arm befestigt ist, und dem Hc-sr04 Gehäuse, da dort wiederum die Schrauben im Weg waren.

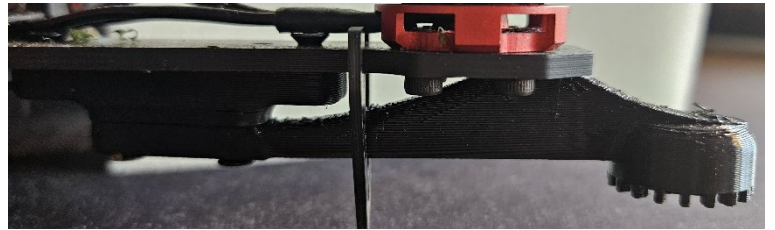


Abbildung 20: Seitenansicht eines Drohnenarms (Quelle: Cedric Kaufmann, Foto, erstellt am 10/09/2024)

Verbindungsstück

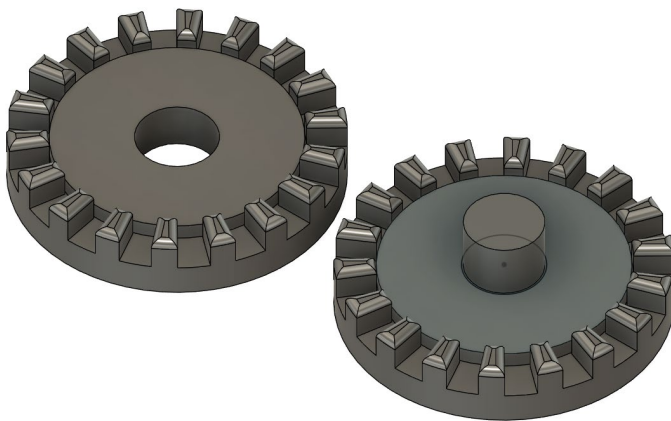


Abbildung 21: 3D Modell der Verbindungsmechanik (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)

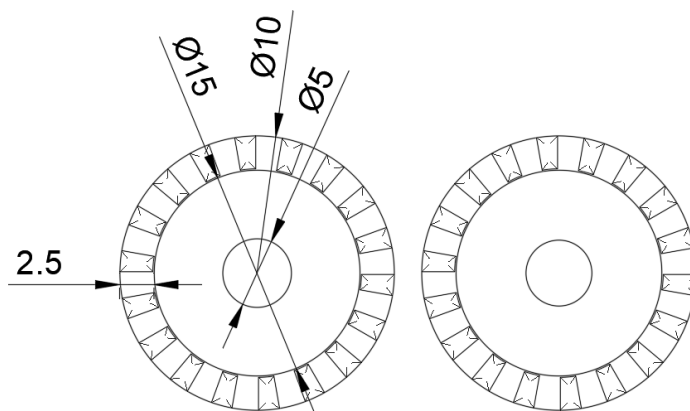


Abbildung 22: Technische Zeichnung der Verbindungsmechanik (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)

Das Verbindungsstück verbindet das Hc-sr04 Gehäuse (siehe Abbildung 17) und des Verbindungsarmes. Die Verbindung muss relativ vielen Kräften standhalten können. Zu den Kräften zählen: Luftwiderstand, Reibung, Vibrationen (welche durch die Rotation der Propeller erzeugt werden) und Luftzirkulation. Deshalb war es wichtig, dass sie sehr stabil sind. Erzielt habe ich das erstens durch ihre Grösse, dadurch konnte ich die einzelnen Zähne robuster gestalten. Und zweitens durch den Zylinder in der Mitte des rechten Teiles, welcher die beiden nochmals besser zusammenhält. Das Konzept mit den Zähnen dient der Freiheit, dass man den Winkel, in welchem die Sensoren messen, frei entscheiden kann und so nicht immer das ganze Stück neu drucken muss. Die Verbindungsmechanik ist schon in den beiden Teilen (Hc-sr04 Gehäuse und Verbindungsarm) fix integriert.

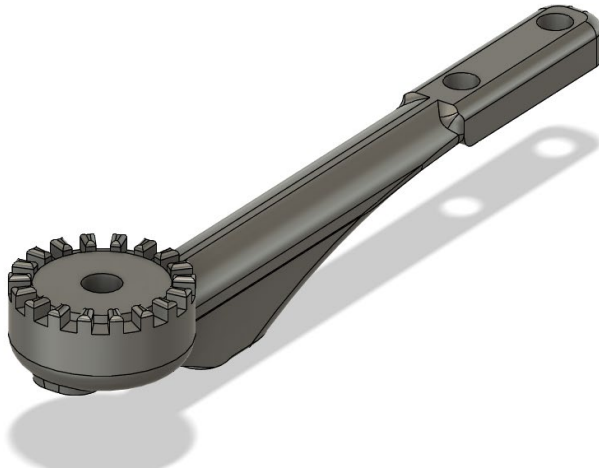
Verbindungsarm

Abbildung 23: 3D Modell des Verbindungsarm (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)

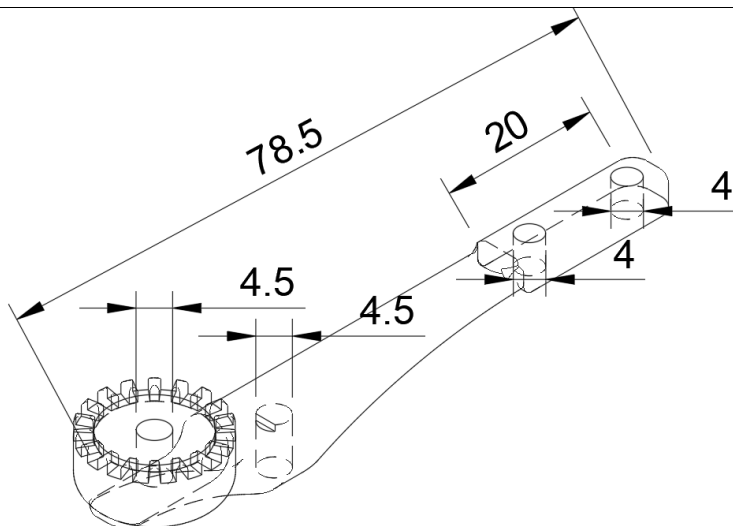
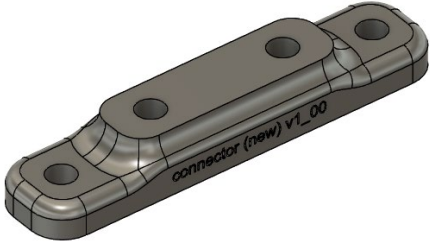
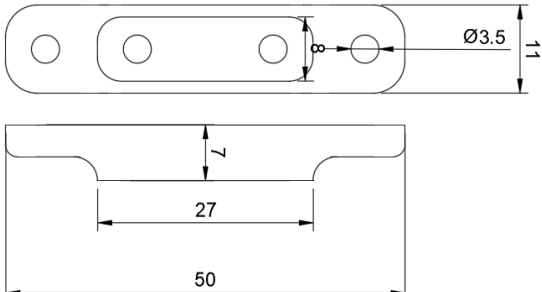
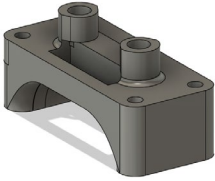
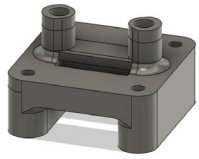


Abbildung 24: Technische Zeichnung des Verbindungsarm (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)

Das mit komplizierteste Teil, welches ich entwickelt habe, müsste wohl der Verbindungsarm sein. Durch die komplexen Strukturen (Drohnenarm, Schrauben der Motoren, genügen Abstand zum Rotor) war die Entwicklung im Vergleich ziemlich anspruchsvoll. Wie Sie sehen, ist das ganze Bauteil abgerundet, dies ist durch den Faktor Luftwiderstand entstanden, Rundungen leisten weniger Widerstand. Zudem sind sie stabiler, durch die bessere Druckverteilung. Der Kronenkranz (oder Zahnrad) ist das auf Seite: 24 erwähnte Verbindungsstück, welches hier in der weiblichen Variante (also ohne Verbindungspin) eingebaut ist. Der Absatz, welcher sich unterhalb des ganzen Teils befindet, ist die Verbindung zwischen Drohnenarm und dem Verbindungsarm. In ihm ist ein Loch eingebaut, in welches eine Schraube eingedreht werden kann (siehe: Abbildung 24). Im hinteren Teil hat es zwei Aussparungen, um den Verbindungsarm noch zusätzlich am Arm anzubringen durch ein weiteres Bauteil (siehe: Abbildung 25)

Verbindungsstück zum Drohnenarm	
 Abbildung 25: 3D Modell des Verbindungsstückes zum Drohnenarm (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	Entwickelt, um den Verbindungsarm an einer zweiten Stelle mit dem Drohnenarm zu verbinden, besteht dieses Bauteil eigentlich nur aus einer Erhebung mit den dazugehörigen Montagelöchern für die Schrauben (bzw. hier als Einlass für Heated inserts¹¹). Die beiden äusseren Löcher dienen dazu, um das Bauteil mit dem Drohnenarm zu verbinden. Durch die nötige Höhe des Verbindungsarms wurden die beiden Montagelöcher in der Mitte um 3 mm erhöht. Wie schon bei den äusseren zwei Löchern werden auch hier Heated inserts angebracht, um ein sicheres und stabiles Gewinde für die Schrauben zu bieten. Die Abrundungen sind in diesem Fall nur aus ästhetischen Gründen entstanden und tragen nicht wirklich der Stabilität bei.
 Abbildung 26: Technische Zeichnung des Verbindungsstückes zum Drohnenarm (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	

Time of flight Halterungen:

Time of flight Halterungen oben und unten		
 Abbildung 27: 3D Modell ToF Halterung unten (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	 Abbildung 28: 3D Modell ToF Halterung oben (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	Die Halterungen für die ToF Sensoren oben und unten sind sehr ähnlich. Aufgebaut aus zwei Stützen (durch welche sie auch auf der Drohne montiert werden). Auf der Oberseite sind zwei Türme, welche dazu dienen, den Sensor mit dem Bauteil zu verbinden. Die Aussparung unterhalb der Türme dient als Weg für die Kabel, sodass sie nicht in den Weg kommen. Der Bogen auf der Unterseite dient rein ästhetischen Gründen und so ist es einfacher für den 3D-Drucker zu drucken, da dieser tendenziell Mühe mit 90° Überhängen hat.

¹¹ Heated Inserts sind metallische Gewindeeinsätze, die in Kunststoffteile eingebettet werden, um eine starke, wärmebeständige Verbindung für Schrauben zu schaffen.

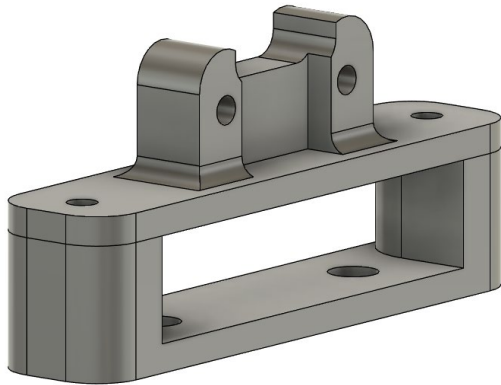
ToF Halterung vorne und hinten

Abbildung 29: 3D Modell ToF Halterung vorne und hinten
(Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)

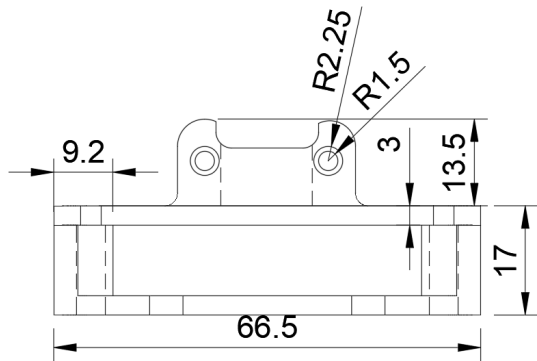


Abbildung 30: Technische Zeichnung ToF Halterung vorne und hinten (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)

Das letzte Bauteil, welches ich entwickelte, ist die Halterung für die Time of flight Sensoren, welche sich vorn und hinten an der Drohne befinden.

Diese Halterung ist aus zwei verschiedenen Elementen aufgebaut. Dem unteren Teil, welcher sich auf der Oberseite der Drohne befindet, und einem Oberteil, welches sich unterhalb der Drohne befindet. Die Stützen sind dazu da, um den Abstand, welcher sich durch den Boden der Drohne bildet, zu überbrücken. Die Form der Stützen wurde spezifisch so gewählt aus denselben Gründen wie bei den anderen Teilen auch (bezüglich den Rundungen).

Auf der flachen Oberseite des oberen Teiles platzierte ich die eigentliche Halterung für den ToF Sensor. Welche sich aus zwei parallelen Stützen, welche durch eine Wand verbunden sind, zusammensetzt. Die Wand ist aus Stabilitätsgründen dort. Der Sensor wird in die dafür eingelassenen Löcher in den Stützen eingeschraubt, hier wieder auf der Rückseite der Stütze ist Platz, um zwei Heated inserts unterzubringen. Die Spitzen der Stützen wurden aus ästhetischen Gründen abgerundet und dienen nicht der Stabilität.

Verwendetes Material für den 3d Druck:

Die Bauteile wurden ursprünglich in PLA ¹² gedruckt, was sich jedoch als zu instabil erwies. Nach längerem Forschen bin ich auf ein Material namens PETG ¹³ gestossen, welches stabiler und auch UV-Resistent ist. Durch das neue Material konnte ich die Langlebigkeit der Einzelteile um ein Vielfaches verbessern. Zudem eignet sich das Material sehr gut, um die Heated inserts darin zu platzieren, da es sehr gut bindet, wenn es einmal geschmolzen ist. Das einzige Problem mit diesem Material war die erhöhte Schwierigkeit beim Drucken, das Material braucht hohe Temperaturen (240° C), welche nicht jeder 3d-Drucker erreichen kann.

¹² PLA ist ein biologisch abbaubarer Kunststoff, der aus nachwachsenden Rohstoffen wie Maisstärke hergestellt wird.

¹³ PETG ist ein robuster, chemikalienbeständiger und leicht zu verarbeitender Kunststoff, der oft im 3D-Druck eingesetzt wird und eine Kombination aus Stärke und Flexibilität bietet.

Design der Leiterplatten

Um die Elektronik meiner Maturaarbeit möglichst effizient und schön zu gestalten, habe ich mich dazu entschieden, eigene Leiterplatten zu entwickeln. Diese ermöglichten es mir, platzsparend Elemente der Drohne wie z. B. das zusätzliche Power Distribution Board¹⁴, welches benötigt wird, um alle Sensoren und den Mikrocontroller mit Strom zu versorgen, unterzubringen und diese auch im Sinne der Effizienz zu optimieren. Die Leiterplatten habe ich mit einer webbasierten Software namens EasyEDA¹⁵ erstellt. Die Software ist frei verfügbar und nutzbar, was sie ideal für meine Zwecke machte. Für meine Maturaarbeit entwickelte ich zwei verschiedene Leiterplatten: Eine Leiterplatte, um den Mikrocontroller unterzubringen und seine Pins (Anschlüsse) einfach erreichbar zu machen, und die zweite Leiterplatte, um die Sensorik der Drohne (welche ich zusätzlich hinzugefügt habe) mit Strom zu versorgen. Die zweite Leiterplatte beherbergt auch zusätzliche Elektronik für die Datenkommunikation zwischen Teensy 4.1 und den Sensoren. Da die Pins des Teensy 4.1 keine 5 Volt Logik vertragen, musste ich noch Widerstände (2k Ohm und 1k Ohm) unterbringen, um die Spannung auf 3.3V zu reduzieren. Um die ganze Logik platzsparend zu machen, habe ich auf der zweiten Leiterplatte auch noch den Multiplexer (Für mehr Informationen über den Multiplexer siehe Seite: 15) untergebracht, da ich dort noch Platz hatte.

1. Die wichtigsten Begriffe für das Verständnis

Um die nächsten Abschnitte besser zu verstehen, muss man über ein gewisses Grundwissen der Elektrotechnik verfügen. Im Folgenden werde ich die wichtigsten Begriffe erklären und erläutern, welche im späteren Verlauf auftauchen können. Beginnen wir mit den Grundlegenden drei Begriffen: Ampere, Volt und Ohm, die in einer speziellen Beziehung zueinanderstehen, welche durch das sogenannte Ohm'sche Gesetz¹⁶ beschrieben wird.

Das Ohm'sche Gesetz:

1. Ampere (I): Stromstärke

Mit Ampere wird die Stromstärke gemessen, also die Menge an elektrischer Ladung, die pro Sekunde durch einen elektrischen Leiter fließt. Das bedeutet, je mehr Ampere in einem Stromkreis fließen, desto mehr elektrische Energie wird transportiert.

Im Grunde gibt es für die drei Begriffe (Ampere, Volt, Ohm) das Wasserschlauch-Beispiel. Für Ampere gilt: Die Menge an Wasser, die pro Sekunde durch den Schlauch fließt, entspricht der Stromstärke in einem elektrischen Stromkreis, also den Ampere.

¹⁴ Unter dem Power Distribution Board versteht man eine Leiterplatte, welche die als Hub für die Energieversorgung der elektrischen Module dient, in meinem Fall ist das power distribution board die Zentrale Einheit, von welcher aus der Strom auf das System verteilt wird.

¹⁵ <https://easyeda.com/>

¹⁶ Durch das Ohm'sche Gesetz wird die Beziehung zwischen Volt (U), Ampere (I) und Widerstand (R) dargestellt. Das Gesetz besagt, dass die Größen in folgender Relation stehen: $U = I \cdot R$. Das bedeutet, dass bei einer konstanten Spannung eine Erhöhung des Widerstands den Stromfluss verringert und umgekehrt.

2. Volt (U): Spannung

Um die elektrische Spannung zu messen, verwendet man die Einheit Volt, welche die Kraft beschreibt, die den elektrischen Strom durch einen Leiter treibt. Das heißt, bei gleichem Widerstand kann durch eine Erhöhung der Spannung der Stromfluss erhöht werden. Mit anderen Worten gibt Volt an, wie viel Energie pro Ladungseinheit zur Verfügung steht.

Im Wasserschlauch-Beispiel erklärt: Die Spannung (Volt) kann mit dem Wasserdruck verglichen werden, der im Schlauch herrscht. Ein höherer Druck (mehr Volt) bedeutet, dass das Wasser (bzw. der Strom in einem elektrischen Schaltkreis) schneller und kräftiger durch den Schlauch gedrückt wird bzw. durch die Leiterbahnen fließt.

3. Ohm (Ω): Widerstand

Ohm gibt an, wie viel elektrischen Widerstand ein Material besitzt, also wie stark ein Material den Stromfluss behindert.

Im Wasserschlauch Beispiel wäre Ohm ein Engpass im Schlauch oder eine Verengung, welche den Effekt hat, dass der Wasserfluss verlangsamt wird. Das heisst, je grösser der Widerstand, desto weniger Strom kann in einer Leiterbahn fließen.

Verbunden werden diese Begriffe durch das Ohm'sche Gesetz, welches die Beziehung zwischen Spannung (U: Volt), Stromstärke (I: Ampere) und Widerstand (R: Ohm Ω) beschreibt. Das Gesetz lautet wie folgt:

$$\text{Volt} = \text{Ampere} * \text{Widerstand}; \text{ In mathematischer Schreibweise wäre dies: } U = I * R$$

Die Bedeutung der Formel

- Wenn die Spannung (U) erhöht wird, wird die Menge an Strom, der fließt erhöht (also I) vorausgesetzt der Widerstand (R) bleibt gleich / konstant.
- Bei gleichbleibender Spannung (U) führte eine Reduktion des Widerstands (R) zu einer Erhöhung des Stroms (I).
- Bei konstanter Spannung (U) führt eine Erhöhung des Widerstands (R) zu einer Verminderung des Stromflusses (I).

Veranschaulichung an einem Beispiel

Wenn der Widerstand hoch ist, aber die Spannung tief, also wenn es eine starke Verengung im Schlauch gibt und der Wasserdruck tief ist, wird nur wenig Wasser fließen (= eine geringe Stromstärke). Anders ist es bei einer hohen Spannung (=hoher Wasserdruck) und niedrigem Widerstand (=kleine Verengung), was zu einer hohen Stromstärke führt (= es fließt viel Wasser).

Weiter wichtige Begriffe:

1. Leiterplatte (PCB)

Eine Leiterplatte, auch Platine bezeichnet, ist eine Platte aus einem Isoliermaterial (Glasfaser oder Epoxidharz), auf welcher die Leiterbahnen verlaufen und elektronische Bauteile montiert werden. Leiterplatten dienen der mechanischen und elektrischen Verbindung der Bauteile, eine Leiterplatte ist bei fast allen elektronischen Geräten das Herzstück, auf welchem alle elektronischen Bauteile befestigt sind.

2. Leiterbahnen

Eine Leiterbahn ist eine elektrische Verbindung auf einer Leiterplatte (= Platine), welche den Stromfluss zwischen verschiedenen Komponenten ermöglicht. Die Resistenz einer Leiterbahn wird durch ihre Breite und Dicke beeinflusst, mit anderen Worten, wie viel Strom eine Leiterplatte sicher transportieren kann. Leiterbahnen bestehen aus Materialien, welche Strom sehr gut und ohne grossen Widerstand leiten können. Beispiele für Materialien, welche bei der Produktion von Leiterplatten eingesetzt werden, wären: Kupfer, Silber oder Gold. Jedoch, da Silber und Gold sehr kostspielig werden können, wird Kupfer mitunter am meisten verwendet.

3. Leistung (Watt, W)

Die Menge an Energie, welche pro Sekunde verbraucht oder übertragen wird, wird mit Watt angegeben. Im Wasserbeispiel wäre das, die Menge an Wasser, welche pro Sekunde aus einem Schlauch mit einem bestimmten Druck fliesst. Um die Leistung (Watt) zu berechnen, muss man die elektrische Spannung (Volt) mit der Stromstärke (Ampere) multiplizieren.

Quellen: (Energis, 2024), (E wie Einfach, 2024), (Peterson, 2020)

2. Elektrischer Schaltplan der Leiterplatten

Mikrocontroller Platte

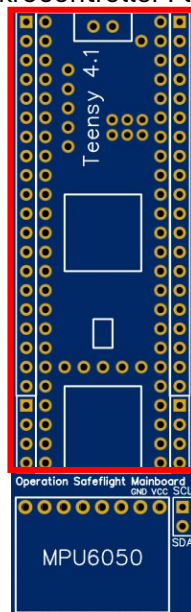


Abbildung 31: Leiterplatte für den Mikrocontroller
(Quelle: Cedric Kaufmann, Screenshot aus EasyEDA,
erstellt am 04/09/2024)

Powerboard mit Multiplexer Halterung

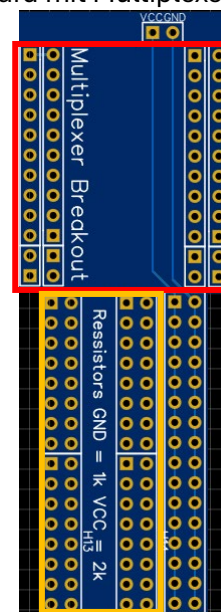


Abbildung 32: Powerboard Leiterplatte (Quelle: Cedric Kaufmann, Screenshot aus EasyEDA, erstellt am
04/09/2024)

Produziert wurden die Leiterplatten mithilfe einer Firma namens JLCPCB¹⁷, welche die Leiterplatten zu einem günstigen Preis herstellt und einem zuschicken. Der ganze Prozess der Entwicklung der Leiterplatten war ziemlich selbsterklärend, bis auf, dass ich zuerst noch lernen musste, wie man Leiterplatten zeichnet. Das nötige Wissen über die elektronischen Schaltkreise hatte ich mir früher schon einmal angeeignet, was dem Prozess half. Im Grundlegenden sind die Einzelteile der Leiterplatten nur Sitze für die verschiedenen elektronischen Module, wie dem Mikrocontroller oder des Multiplexer (siehe rote Kästen).

¹⁷ <https://jlcpcb.com/>

3. Probleme und Überlegungen beim Entwickeln der Leiterplatten

1. Regulierung der Spannung (Voltage Divider)

Da die Pins des Teensy 4.1 auf einer 3.3 Volt Logik funktionieren, die Energieversorgung, welche durch das power distribution board bereitgestellt wird, aber auf 5 Volt basiert. War ich gezwungen, die Voltanzahl zu reduzieren, um die Pins vor der erhöhten Spannung, welche auf den Kommunikationslinien liegt, zu schützen. Durch das Ohm'sche Gesetz wissen wir, dass bei gleichbleibender Stromstärke der Widerstand erhöht werden muss, um die Spannung zu reduzieren. Diese Reduktion wird durch sogenannte Resistors (=Widerstände) erzielt. Die Widerstände wurden Platzeffizient auf dem power distribution board (Abbildung 32) im orangenen Kasten untergebracht. Um zu berechnen, wie viel Widerstand ich verwenden musste, wurden folgende Berechnungen (welche hier im groben Überblick erklärt werden) verwendet.

Die grundlegende Formel für die Berechnung einer Ausgangsspannung V_{Out} mit einem Spannungsteiler ist die Folgende:

$$V_{Out} = V_{In} \times \frac{R_2}{R_2 + R_1}$$

Dabei gilt Folgendes:

- V_{Out} : Ausgangsspannung in Volt
- V_{In} : Eingangsspannung in Volt
- R_1 und R_2 sind die Widerstände¹⁸ in Ohm

Aus der Formel ersichtlich wird, dass die Eingangsspannung V_{In} über die beiden in Reihe geschalteten Widerstände R_1 und R_2 aufgeteilt wird. Wobei sich die Spannung proportional zu den Widerstandswerten aufteilt. Um die Spannung zu skalieren, braucht man den Faktor, $\frac{R_2}{R_2 + R_1}$ welcher die Eingangsspannung auf die gewünschte Ausgangsspannung skaliert. Dieser Faktor gibt an, wie viel der Eingangsspannung am Widerstand R_2 abfällt und somit die gewünschte Ausgangsspannung bildet.

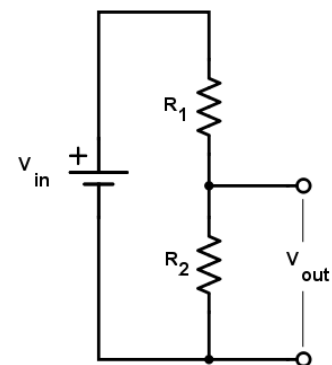


Abbildung 33: Voltage divider circuit (Quelle: All about Circuits, <https://www.allaboutcircuits.com/tools/voltage-divider-calculator/>, abgerufen am 07/09/2024)

2. Anwendung der Formel, um 5V auf 3.3V zu skalieren

Für meine Arbeit musste ich eine Eingangsspannung von 5V auf eine Ausgangsspannung von 3.3V skalieren. Da die Pins des Teensy 4.1, welchen ich benutze, nicht auf 5V ausgelegt sind. Im Folgenden stelle ich Ihnen die Berechnungen vor, welche ich getätigt habe.

1. Die bekannten Werte (Eingangsspannung und Ausgangsspannung) einsetzen:

Für die Eingangsspannung (V_{In}) wird der Wert von 5V eingesetzt und für die Ausgangsspannung (V_{Out}) der Wert von 3.3V.

$$3.3V = 5V \times \frac{R_2}{R_2 + R_1}$$

¹⁸ Der Wert eines Widerstandes wird in Ω (Ohm) angegeben. Ohm ist die Einheit des elektrischen Widerstands und wird verwendet, um den Widerstand zu messen, den ein elektrischer Leiter dem Stromfluss entgegensetzt.

2. Die Gleichung nach dem Verhältnis der Widerstände umstellen:

$$\frac{3.3V}{5V} = \frac{R_2}{R_2 + R_1} = 0.66$$

3. Berechnung der Widerstandswerten:

Aus der Formel von Schritt 2 ergibt sich, dass der Widerstand R_2 66 % des Gesamtwiderstandes von $R_1 + R_2$ ausmachen muss. Und es wird ersichtlich, dass $R_1 \approx 1 \text{ k}\Omega$ und $R_2 \approx 2 \text{ k}\Omega$ sein muss. Wenn wir dies in die Formel einsetzen, erhalten wir:

$$\frac{R_2}{R_2 + R_1} = \frac{2 \text{ k}\Omega}{2 \text{ k}\Omega + 1 \text{ k}\Omega} = \frac{2}{3} \approx 0.66$$

4. Überprüfung durch Einsetzen in die Original Formel:

$$\begin{aligned} V_{Out} &= V_{In} \times \frac{R_2}{R_2 + R_1} \\ &= 3.3V = 5V \times \frac{R_2}{R_2 + R_1} \\ &= 3.3V = 5V \times \frac{2 \text{ k}\Omega}{2 \text{ k}\Omega + 1 \text{ k}\Omega} \\ &= 3.3V = 5V \times 0.66 \end{aligned}$$

Man kann erkennen, dass die Formel mit den berechneten Widerständen aufgeht und durch die Widerstände R_1 und R_2 die Eingangsspannung von 5V auf die gewünschte 3.3V reduziert wurde.

3. Platzierung der Widerstände

1. Für die ToF (Time of flight) Sensoren

Die theoretischen Berechnungen von vorhin sind in der Realität viel schwieriger anzuwenden. Beginnen wir mit der Platzierung für die ToF Sensoren: Da die ToF Sensoren zusätzlich durch einen Multiplexer am Teensy 4.1 angeschlossen sind, verkompliziert sich das ganze Problem; im Folgenden ist das Verbindungsschema gezeigt.

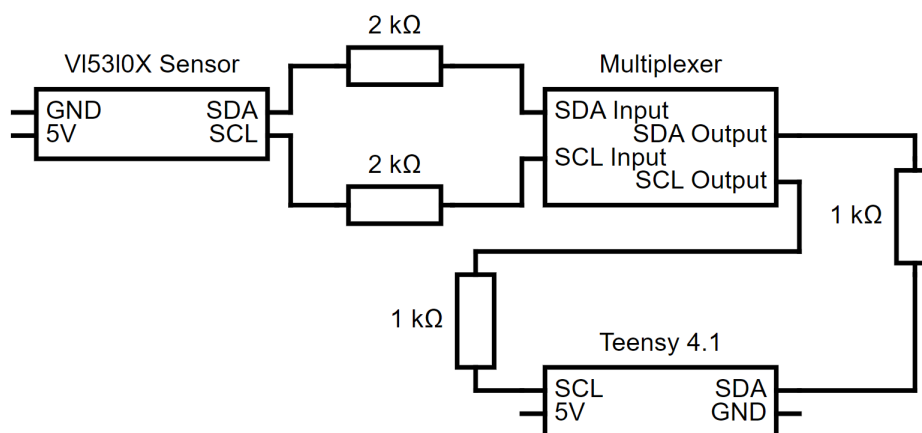


Abbildung 34: Spannungsverteiler ToF Sensoren (Quelle: Cedric Kaufmann, Circuit Diagramm Web Editor, erstellt am 12/09/2024)

In Worten bedeutet das: Zwischen dem Ausgangspin des ToF Sensors (gleich für SDA oder SCL) muss ein 2k Ohm Resistor (Widerstand) platziert werden, welcher dann die Verbindung ausgleicht, welche zum Eingang des Multiplexeres führt. Durch den Voltage Divider muss anhand der zuvor ausgerechneten Widerstände noch ein 1k Ohm Resistor (Widerstand) in der Leitung zwischen dem Multiplexer und dem SDA / SCL Pin des Teensy 4.1 installiert werden.

2. Für die Hc-sr04 Sensoren

Für die Hc-sr04 Sensoren sieht das ganze schon einfacher aus. Der Trig Pin der Sensoren wird durch die 3.3V des Teensy 4.1 gesteuert und ist demnach kein Problem. Das Problem besteht im Echo-Pin, welcher durch die 5V-Logik des Sensors gesteuert wird. Deshalb wird die gleiche Resistor Kombination aus $R_1 = 1k\text{ Ohm}$ und $R_2 = 2k\text{ Ohm}$ verwendet, um die Spannung von 5V auf 3.3V zu reduzieren. Da hier nur eine Leitung zur Verfügung steht, wird der Spannungsverteiler demnach komplizierter. Das Verbindungsschema für den Hc-sr04 Sensor sieht wie folgt aus:

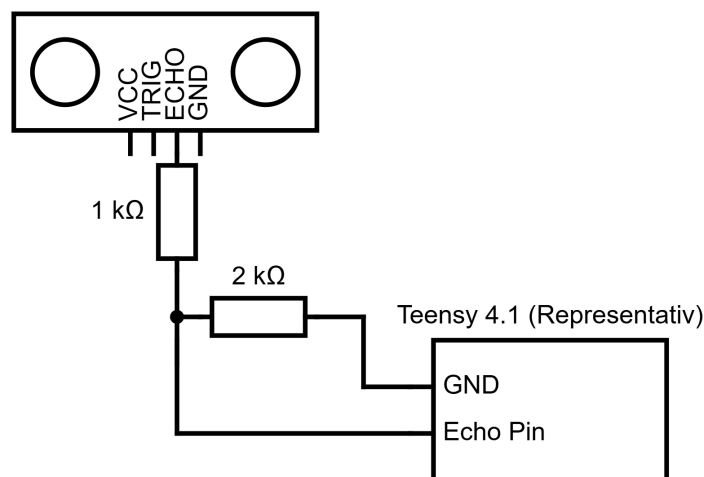


Abbildung 35: Spannungsverteiler Hc-sr03 (Quelle: Cedric Kaufmann, Circuit Diagramm Web Editor, erstellt am 12/09/2024)

Man kann erkennen, dass eine Verzweigung gebraucht wird. Diese Ausgleichsleitung wird gebraucht, um die Spannung konstant und sicher zu regulieren. Zuerst wird der Echo Pin des Hc-sr04 Sensor mit einem 1k Ohm Resistor verbunden, diese Leitung führt zum Echo Pin auf dem Teensy 4.1. Jedoch wird irgendwo auf dieser Leitung dann eine Ausgleichsleitung angebracht, welche zum GND des Teensy 4.1 führt. Auf dieser Leitung wird dann der R_2 Resistor angebracht, um den Spannungsverteiler zu vervollständigen. Da die Drohne mit 4 Hc-sr04 Sensoren operiert wird, wird die Ausgleichsleitung aus Effizienzgründen zusammengefasst. Das heisst: Die einzelnen Kabel werden vor dem GND Pin zusammengeführt, sodass nur ein GND Pin des Teensy 4.1 verwendet wird und nicht 4, die Resistoren bleiben aber natürlich auf den Leitungen.

Quellenangabe für die Berechnungen und die Platzierung der Widerstände: (All about circuits, 2024), (JIMBLOM, 2024), (Electronics Tutorials, 2024), (Astels, 2024), (Arduino Forum, 2024)

Code Vertiefung

Im Rahmen meiner Maturaarbeit musste ich mich auch dem Programmieren widmen. Der Teil meiner Arbeit, welcher auf Code basiert, ist zuständig für die Kommunikation zwischen dem separat hinzugefügten Mikrocontroller und dem Flightcontroller der Drohne. Zusätzlich ermöglicht der Code die Kommunikation zwischen den Sensoren und dem Mikrocontroller; dies ist wichtig für das Kollisionspräventionssystem. Im Weiteren werde ich mich vertieft mit dem Teil des Codes beschäftigen, welcher die PPM-Signale ausliest und wieder weiterleitet. Dies ist der Teil, auf welchen ich besonders stolz bin, da es ein neu entwickeltes System ist, welches so noch nicht existierte.

Der Code ausschnitt ist ein System, welches ich selbst entwickelte, um wie eben erwähnt die Funksignale, welche die Drohne von der Fernbedienung erhält, auszulesen und später wieder weiterzuleiten. Die weitergeleiteten Signale sind ebenfalls wieder im Format PPM, sodass der Flightcontroller diese wieder normal interpretieren kann. Dies zu programmieren, dauerte viel länger, als ich eigentlich gedacht hatte, da ich auf verschiedenste unerwartete Probleme stiess. Zu den Problemen zählten, das Timing der einzelnen Signale war nicht korrekt, die Signale waren umgekehrt, das heisst: immer, wenn das Signal auf «Hoch» sein sollte, war es auf «Tief» und umgekehrt und schlussendlich war die grösste Challenge das Funkprotokoll so zu verstehen, dass ich es in Code umwandeln konnte

Setup der Funktion:

Aufgerufen wird die Funktion «readPPM()» so im Code nie, dies wird erreicht dadurch, dass sie auf einen Interrupt Pin des Teensy 4.1 (alle Digital Pins des Teensy 4.1 sind Interrupt fähig) gebunden wird. Das heisst jedes Mal, wenn dieser Pin eine Änderung feststellt, wird die Funktion aufgerufen. In C++ wird dies so gemacht:

```
void ppmSetup()
{
    // ppm setup
    pinMode(PPM_PIN, INPUT);
    attachInterrupt(digitalPinToInterrupt(PPM_PIN), readPPM, FALLING);
    pinMode(PPM_PIN_OUT, OUTPUT);
    digitalWrite(PPM_PIN_OUT, LOW);
}
```

Abbildung 36: Setup für die PPM relevanten Funktionen (Quelle: Cedric Kaufmann, Visual Studio, erstellt am 21/09/24)

Zuerst muss der PPM_PIN als Input Pin markiert werden, das heisst er sendet keine Signale, sondern empfängt sie. Dann, mit der Funktion: attachInterrupt, wird der PPM_PIN mit einer Interrupt-Funktion ausgestattet. Die Funktion besitzt drei Parameter:

1. **digitalPinToInterrupt():** Mit dieser Funktion wird ein Digital Pin des Teensy 4.1 zu einem Interrupt fähigen Pin umgewandelt

2. **readPPM**: Im zweiten Parameter wird definiert welche Funktion aufgerufen werden soll, wenn ein Interrupt festgestellt wird.
3. **FALLING**: Dies ist der Trigger-Modus des Interrupt Pins, das heisst was erfüllt werden muss, sodass die Funktion readPPM ausgeführt wird. Im Fall von Falling wäre das: Der Interrupt wird ausgelöst, wenn das Signal am angegebenen Pin von HIGH nach LOW wechselt.

Die letzten zwei Zeilen sind für das Senden der PPM-Signale nicht von Relevanz.

Erklärung der Funktion:

Variablen

```
static uint32_t lastTime = 0;
uint32_t currentTime = micros(); // Current time in microseconds
```

Abbildung 37: Variablen Definition readPPM Funktion (Quelle: Cedric Kaufmann, Visual Studio Code, erstellt am 21/09/24)

Die Variable «lastTime» speichert den Zeitpunkt des letzten gemessenen Pulses (in Mikrosekunden). Sie wird zu Beginn auf 0 gesetzt, da am Anfang des Signals noch kein Puls vorangeht. Durch das fortlaufende Aktualisieren von «lastTime» wird sichergestellt, dass immer das nächste Zeitintervall gemessen wird. Durch die Variable «currentTime» wird immer die aktuelle Zeit in Mikrosekunden gespeichert. Die Funktion «micros()» startet die Zeitmessung.

Berechnung der Zeitdifferenz

Die Berechnung der Zeitdifferenz der letzten Beiden Pulsen wird durch folgende Zeilen Code gemacht:

```
uint32_t interval = currentTime - lastTime;
lastTime = currentTime;
```

Abbildung 38: Berechnung der Zeitdifferenz (PPM Signale) (Quelle: Cedric Kaufmann, Visual Studio Code, erstellt am 21/09/24)

Durch die Substruktion der «lastTime» von der «currentTime» wird das aktuelle Zeitintervall bestimmt. Danach um das nächste bestimmen zu können muss die «lastTime» mit der «currentTime» gleichgesetzt werden.

Syncpuls Check

Bei jedem Zeitintervall muss geprüft werden, ob dieses Zeitintervall nicht den Synchronisationspuls darstellt; dies wird erreicht, indem die Dauer des Zeitintervalls überprüft wird. Ist dies länger als die vorgegebene Zeit für die maximale Dauer des Zeitintervalls weiss das Programm, dass dies der Synchronisationspuls ist und setzt den «currentChannel» auf null, durch diesen Channel wird später dann auch die Zuweisungen zu den einzelnen Variablen (welche die Kanäle repräsentieren) gewährleistet.

```
if (interval >= 3000)
{
    // Sync pulse detected (interval longer than a frame period, e.g., 3 ms)
    currentChannel = 0;
}
```

Abbildung 39: Synchronisationspuls Check PPM (Quelle: Cedric Kaufmann, Visual Studio Code, erstellt am 21/09/24)

Zuweisung zu den Einzelnen Kanälen

```
else
{
    switch (currentChannel)
    {
        case 0:
            channel1 = interval;
            break;
        case 1:
            channel2 = interval;
            break;
        case 2:
            channel3 = interval;
            break;
        case 3:
            channel4 = interval;
            break;
        case 4:
            channel5 = interval;
            break;
        case 5:
            channel6 = interval;
            break;
        case 6:
            channel7 = interval;
            break;
        case 7:
            channel8 = interval;
            break;
    }
    currentChannel++;
}
```

Durch die Verwendung der switch-Anweisung wird sichergestellt, dass die Intervalle den richtigen Variablen zugordnet werden. Die Variable «currentChannel» gibt an, welcher Kanal gerade aktiv ist. Je nachdem, welcher Kanal aktuell ausgewählt ist, wird der Wert in die entsprechende Kanalvariable gespeichert (siehe Abbildung 40).

Die Nutzung von «currentChannel» ermöglicht eine sequenzielle Zuordnung der Intervalle zu den verschiedenen Kanälen, ohne dass eine explizite Schleife verwendet wird. Das Prinzip bleibt jedoch ähnlich: Die Kanäle werden nacheinander adressiert und aktualisiert, was eine effiziente und organisierte Handhabung der PPM-Daten ermöglicht.

Abbildung 40: Zuweisung zu den Kanälen (Quelle: Cedric Kaufmann Visual Studio Code, erstellt am 21/09/24)

Reflexion

Die Herangehensweise und das Konzept der Entwicklung eines Produkts sind ebenso wichtig wie das schlussendliche Resultat. Es kann immer wieder zu Problemen oder Herausforderungen während der Entwicklung kommen, die bewältigt werden müssen, um voranzukommen. Es ist normal, dass nicht immer alles nach Plan verläuft, und rückblickend würde man verschiedene Arbeitsprozesse anders gestalten oder gar neue Methoden anwenden. In diesem Abschnitt wird kritisch auf den Arbeitsprozess und die Entwicklung zurückgeblickt und eine Beurteilung dessen vorgenommen.

Betrachtet man das entstandene Produkt, wird deutlich, dass es sich um ein Konzept handelt, das nicht dazu ausgelegt ist, vollständig funktionstüchtig zu sein. Von Beginn an war klar, dass es aus zeitlichen und finanziellen Gründen nicht möglich wäre, ein voll funktionsfähiges Produkt zu entwickeln. Die strukturelle Planung der Arbeit hat sich im Verlauf des Arbeitsprozesses immer wieder bewährt. Die klare Herangehensweise ermöglichte es, das gesamte Projekt strukturiert anzugehen, zunächst das notwendige Fachwissen zu erwerben und dieses später im Entwicklungsprozess effizient anzuwenden. Hingegen stellte sich die zeitliche Planung der einzelnen Arbeitsschritte als grosse Herausforderung heraus. Das Aneignen der theoretischen Grundlagen über die verschiedenen Funkprotokolle und Sensoren sowie das Vertiefen und Erlernen einer bis dahin nur oberflächlich beherrschten Programmiersprache liessen den eigens erstellten Zeitplan nicht aufgehen. Die Erarbeitung der Theorie nahm viel mehr Zeit in Anspruch als ursprünglich gedacht, da es sich um ein komplexes Thema handelte, was bei der Planung unterschätzt wurde. Dadurch entstand ein Rückstand, der jedoch wieder aufgeholt werden konnte. Verantwortlich dafür waren die entwickelten 3D-gedruckten Elemente des Projekts, die durch das umfassende Vorwissen im Bereich CAD effizient und problemlos entwickelt werden konnten. Da bereits im Vorhinein die theoretischen Grundlagen über Luftströme und Vibrationen in mechanischen Systemen erarbeitet worden waren, konnten diese Erkenntnisse flussend in die Entwicklung der Bauteile einfließen.

Durch die ursprünglich nicht geplante Entwicklung eigener Leiterplatten geriet der Zeitplan erneut unter Druck, da es sich hierbei um ein zuvor noch nie betretenes Gebiet handelte, dessen Erlernen sich als schwieriger erwies als erwartet. Rückblickend würden viele Entwicklungsentscheidungen heute anders getroffen werden, was beispielsweise die vermehrte Verwendung von Verbindungskabeln zwischen den einzelnen Komponenten hätte verringern können. Allerdings war das notwendige Fachwissen zum Zeitpunkt der Finalisierung der Leiterplatten noch nicht vorhanden und wurde erst später durch andere Nebenprojekte erlangt. Diese Ineffizienz wird daher nicht als Rückschlag angesehen, sondern vielmehr als Lernprozess für zukünftige Projekte, da sie mir den Einstieg in dieses Fachgebiet ermöglichte.

Ein beträchtlicher Teil der Arbeit fand hinter dem Rechner statt. Das Programmieren des Teensy 4.1 wurde mit der richtigen Herangehensweise angegangen: Zuerst wurde überprüft, ob alle verwendeten Komponenten miteinander kompatibel sind, was sich als nützlich erwies, als festgestellt wurde, dass eine Library, die verwendet werden sollte, nicht auf der ARM-Struktur des Teensy 4.1 funktionierte und eine Alternative gefunden werden musste. Das eigentliche Programmieren nahm mehr Zeit in Anspruch als ursprünglich gedacht, da die PPM-Signale invertiert waren, was erst durch die Betrachtung mit einem Oszilloskop ersichtlich wurde.

Das Empfangen und Senden der PPM-Signale wurde sehr effizient gelöst und funktioniert einwandfrei, was dank der guten theoretischen Grundlage möglich war. Zu spät wurde erkannt, dass der Mikrocontroller auf einer anderen Spannungsebene arbeitet als die restlichen Komponenten

der Drohne, was zur Implementierung eines expliziten Powermanagementsystems führte, das Spannungsausgleichsleitungen, Spannungsregler und verschiedene Schutzsysteme umfasste. Dieser Prozess verlief trotz des Zeitdrucks reibungslos, da das notwendige Fachwissen bereits durch verschiedene Projekte erworben worden war und mir ein Elektrotechniker bei Fragen zur Seite stand.

Die Zusammensetzung der einzelnen Komponenten (3D-gedruckte Teile, Sensoren, Powermanagement und Code) erfolgte durch die Implementierung des Powermanagementsystems in einem dafür nicht vorgesehenen Zeitrahmen und wurde dementsprechend schnell ausgeführt.

Zusammenfassend würde ich sagen, dass ich mit dem Konzept der Drohne zufrieden bin. Die einzelnen Komponenten wurden sorgfältig erarbeitet, und trotz der Fehleinschätzung des Zeitaufwands wurde alles erreicht, was ich mir vorgenommen hatte. Besonders stolz bin ich auf das entwickelte System zum Abfangen der PPM-Signale, welches in dieser Form noch nirgends implementiert wurde.

Falls ich wieder ein vergleichbares Produkt entwickeln sollte, würde ich zuerst klar definieren, was gemacht werden muss, um Fehler wie beim Powermanagementsystem zu vermeiden. Zudem wurde deutlich, dass das gesamte Projekt etwas zu umfangreich für eine einzelne Person war. In Zukunft würde ich die Anforderungen an mich selbst und das Projekt reduzieren, um den Zeitdruck zu verringern, der in dieser Arbeit vor allem durch den schriftlichen Teil entstanden ist.

Unter Zeitdruck war die Verwendung künstlicher Intelligenz hilfreich, da sie mir entscheidende Ideen für die Entwicklung des Codes sowie für die Gestaltung der schriftlichen Arbeit lieferte. Es ist jedoch wichtig zu betonen, dass alles Geschriebene (sowohl Code als auch Text) von mir verfasst wurde. Die KI wurde lediglich für die Ideenfindung und zur Erstellung eines Grobplans für die Entwicklung des Codes verwendet. Die Programmierung wurde von mir durchgeführt, getestet und überprüft, wobei ich im letzten Schritt bei etwaigen Problemen die KI um Rat fragte, um mögliche Fehler zu identifizieren.

Disclaimer

Das entworfene Kollisionspräventionssystem, welches zusätzlich auf die Drohne montiert wurde und auf dem Teensy 4.1 läuft, wurde als eine unterstützende Technologie entwickelt, um den Piloten beim Erlernen des Fliegens zu entlasten und den sicheren Betrieb der Drohne zu vereinfachen. Es dient dazu, allfällige Kollisionen der Drohne zu vermeiden; dazu werden Sensordaten in Echtzeit analysiert und potenzielle Gefahren (Objekte, Wände, Boden) erkannt. Es ist trotz des Kollisionspräventionssystems wichtig zu betonen, dass der Pilot zu jeder Zeit die absolute Kontrolle über sein Fluggerät (hier eine Drohne) behalten muss. Das System greift nicht autonom in die Steuerung der Drohne ein, sondern manipuliert lediglich die PPM-Signale, welche für die Kommunikation zwischen Drohne und Pilot genutzt werden, um den Piloten in kritischen Situationen zu unterstützen.

Es wird ausdrücklich darauf hingewiesen, dass das entwickelte Kollisionspräventionssystem keine garantierte Sicherheit bieten kann. Obwohl es darauf ausgelegt ist, bei der Vermeidung potenzieller Kollisionen zu unterstützen, kann keine Garantie gegeben werden, dass alle Hindernisse korrekt erkannt und vermieden werden. Das System kann durch verschiedenste Faktoren wie Wetterbedingungen, technische Störungen, nicht erkannte Objekte oder Signalverlust beeinträchtigt werden.

Der Pilot trägt weiterhin die alleinige Verantwortung für den sicheren Betrieb der Drohne, einschliesslich der notwendigen Reaktionen auf unerwartete Situationen. Die unterstützende Funktion des Systems soll dem Piloten lediglich als Hilfsmittel dienen, ersetzt jedoch nicht die erforderliche Aufmerksamkeit und das aktive Eingreifen des Piloten.

Vom Gebrauch der Drohne wird kräftig abgeraten, da es sich um ein Konzept handelt und dieses noch kein fertig ausgereiftes Produkt ist. Dazu wären vermehrte Tests und Sicherheitsmassnahmen erforderlich, welche nicht in den Kapazitäten des Entwicklers liegen. Bitte beachten Sie diese Warnung und gehen Sie vernünftig mit dem konzeptuellen Produkt um.

Für allfällige Funktionsuntüchtigkeiten des Systems wird keine Verantwortung übernommen, und es wird stark betont, nicht auf das Kollisionspräventionssystem zu vertrauen, welches aus verschiedensten Gründen nicht funktionieren kann. Wird diese Warnung missachtet, übernimmt der Hersteller des Systems keine Haftung für allfällige Schäden oder Verletzungen. Mit dem Lesen dieses Disclaimers akzeptieren Sie diese Bedingungen und entbinden den Hersteller von allen Konsequenzen.

Für weitere Richtlinien siehe: <https://www.bazl.admin.ch/bazl/de/home/drohnen.html>

Verweise und Links

GitHub: <https://github.com/gitKinsey/Operation-SafeFlight>

Google Docs: https://drive.google.com/drive/folders/1NIOPSiSSpo_FH8sGSdHaMw2L9DiK-XHc8?usp=sharing

Anhang

Abbildungsverzeichnis:

Abbildung 1: Aufbau PlatformIO (Quelle: Screenshot von Visual Studio Code)	6
Abbildung 2: Aufbau einer PlatformIO Datei (Quelle: Screenshot von Visual Studio Code)	7
Abbildung 3: Ultraschall Sensor Hc-sr04 Pinout (Quelle: David Watson, https://www.theengineeringprojects.com/2018/10/introduction-to-hc-sr04-ultrasonic-sensor.html , abgerufen am 21/07/2024)	9
Abbildung 4: Ablauf der Ultraschall-Entsendung (Quelle: How to Mechatronics, https://howto-mechatronics.com/tutorials/arduino/ultrasonic-sensor-hc-sr04/ , abgerufen am 21/07/2024)	9
Abbildung 5: Library in der Arduino IDE installieren (Quelle: Cedric Kaufmann, Arduino IDE, erstellt am 13/09/2024)	10
Abbildung 6: Programmierung Hc-sr04 ohne Library (Quelle: Screenshot von Visual Studio Code)	11
Abbildung 7: Programmierung Hc-sr04 mit NewPing Library (Quelle: Screenshot von Visual Studio Code)	11
Abbildung 8: VL53L0X Sensor (Quelle: Botland, https://botland.de/eingestellte-produkte/10315-vl53l0x-flugzeit-i2c-abstandssensor-adafruit-3317-5904422315481.html , abgerufen am 31/08/24)	13
Abbildung 9: Funktionsweise eines VL53L0X Sensors (Quelle: Piktogramm aus Word, Cedric Kaufmann, erstellt am 01/09/2024)	13
Abbildung 10: Programmierung eines VL53L0X Sensors (Quelle: Codeausschnitt von Visual Studio Code, Cedric Kaufmann, erstellt am 01/09/2024)	14
Abbildung 11: Multiplexer (TCA9548A) (Quelle: Baastelgarage.ch, https://www.bastelgarage.ch/8-channel-i2c-multiplexer-tca9548a , abgerufen am 13/09/2024)	15
Abbildung 12: Pinout Teensy 4.1 (Quelle: PJRC, https://www.pjrc.com/store/teensy41.html , abgerufen am 17/09/24)	16
Abbildung 13: Aufbau eines PWM-Signal (Quelle: BYJU'S, https://byjus.com/physics/pulse-width-modulation/ , abgerufen am 08/08/2024)	18
Abbildung 14: PPM-Signal (Quelle: Arduino Forum, https://forum.arduino.cc/t/generate-ppm-pulses-for-radio-control/616442 , abgerufen am 21/09/24)	20
Abbildung 15: 3D Modell des Deckels für die Hc-sr04 Sensoren (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	22
Abbildung 16: Technische Zeichnung des Deckels für die Hc-sr04 Sensoren (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	22
Abbildung 17: 3D Modell der Vorderseite des Hc-sr04 Gehäuse (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	23
Abbildung 18: 3D Modell der Rückseite des Hc-sr04 Gehäuse (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	23
Abbildung 19: Technische Zeichnung des Gehäuse für die Hc-sr04 Sensoren (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	23
Abbildung 20: Seitenansicht eines Drohnenarms (Quelle: Cedric Kaufmann, Foto, erstellt am 10/09/2024)	24
Abbildung 21: 3D Modell der Verbindungsmechanik (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	24
Abbildung 22: Technische Zeichnung der Verbindungsmechanik (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	24

Abbildung 23: 3D Modell des Verbindungsarm (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	25
Abbildung 24: Technische Zeichnung des Verbindungsarm (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	25
Abbildung 25: 3D Modell des Verbindungsstückes zum Drohnenarm (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	26
Abbildung 26: Technische Zeichnung des Verbindungsstück zum Drohnenarm (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	26
Abbildung 27: 3D Modell ToF Halterung unten (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	26
Abbildung 28: 3D Modell ToF Halterung oben (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	26
Abbildung 29: 3D Modell ToF Halterung vorne und hinten (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	27
Abbildung 30: Technische Zeichnung ToF Halterung vorne und hinten (Quelle: Cedric Kaufmann, Fusion 360, erstellt am 10/09/2024)	27
Abbildung 31: Leiterplatte für den Mikrocontroller (Quelle: Cedric Kaufmann, Screenshot aus EasyEDA, erstellt am 04/09/2024)	30
Abbildung 32: Powerboard Leiterplatte (Quelle: Cedric Kaufmann, Screenshot aus EasyEDA, erstellt am 04/09/2024)	30
Abbildung 33: Voltage divider circuit (Quelle: All about Circuits, https://www.allaboutcircuits.com/tools/voltage-divider-calculator/ , abgerufen am 07/09/2024)	31
Abbildung 34: Spannungsverteiler ToF Sensoren (Quelle: Cedric Kaufmann, Circuit Diagramm Web Editor, erstellt am 12/09/2024)	32
Abbildung 35: Spannungsverteiler Hc-sr03 (Quelle: Cedric Kaufmann, Circuit Diagramm Web Editor, erstellt am 12/09/2024)	33
Abbildung 36: Setup für die PPM relevanten Funktionen (Quelle: Cedric Kaufmann, Visual Studio, erstellt am 21/09/24)	34
Abbildung 37: Variablen Definition readPPM Funktion (Quelle: Cedric Kaufmann, Visual Studio Code, erstellt am 21/09/24)	35
Abbildung 38: Berechnung der Zeitdifferenz (PPM Signale) (Quelle: Cedric Kaufmann, Visual Studio Code, erstellt am 21/09/24)	35
Abbildung 39: Synchronisationspuls Check PPM (Quelle: Cedric Kaufmann, Visual Studio Code, erstellt am 21/09/24)	35
Abbildung 40: Zuweisung zu den Kanälen (Quelle: Cedric Kaufmann Visual Studio Code, erstellt am 21/09/24)	36

Tabellenverzeichnis

Tabelle 1: Technische Daten Hc-sr04	8
Tabelle 2: Technische Daten VL53L0X Sensor	12
Tabelle 3: Technische Daten Multiplexer	15

Literaturverzeichnis

Adafruit. (01. September 2024). *Adafruit VL53L0X*. Von https://github.com/adafruit/Adafruit_VL53L0X abgerufen

- All about circuits . (2024, September 06). *All about circuits* . Retrieved from Voltage divider calculator: <https://www.allaboutcircuits.com/tools/voltage-divider-calculator/>
- All about circuits. (07. September 2024). *Voltage Divider Calculator*. Von <https://www.allaboutcircuits.com/tools/voltage-divider-calculator/> abgerufen
- Arduino Forum. (11. September 2024). *Arduino voltage Divider*. Von <https://forum.arduino.cc/t/arduino-voltage-divider/636659> abgerufen
- Astels, D. (11. September 2024). *Adafruit*. Von Voltage Dividers : <https://learn.adafruit.com/getting-to-know-the-555/voltage-dividers> abgerufen
- Baumer Passion for Sensors. (13. September 2024). *Ultraschallsensoren*. Von https://www.baumer.com/ch/de/service-support/funktionsweise/funktionsweise-von-ultraschallsensoren/a/Know-how_Function_Ultrasonic-sensors#:~:text=Oberfl%C3%A4che,0%2C2%20mm%20nicht%20%C3%BCbersteigt. abgerufen
- Digikey. (20. August 2024). *Grundlagen zu Ultraschallsensoren*. Von <https://www.digikey.de/de/articles/understanding-ultrasonic-sensors#:~:text=Ultraschallsensoren%20bieten%20zwar%20viele%20Vorteile%20und%20Vorz%C3%BCge%20gegen%C3%BCber,die%20Form%20oder%20Farbe%20eines%20Objekts.%20Weitere%20Elemente> abgerufen
- E wie Einfach. (06. September 2024). *Watt, Volt und Ampere: Wissenswertes rund ums Thema Energie*. Von <https://www.e-wie-einfach.de/strom/ratgeber/watt-volt-und-ampere-wissenswertes-rund-ums-thema-energie> abgerufen
- Electronics Tutorials. (11. September 2024). *Electronics Tutorials*. Von Voltage Divider: <https://www.electronics-tutorials.ws/dccircuits/voltage-divider.html> abgerufen
- Energis. (06. September 2024). *Stromeinheiten Watt, Volt und Ampere einfach erklärt*. Von https://energis.de/ratgeber/strom/watt_volt_ampere abgerufen
- Ewald, W. (15. September 2024). *Wolles ELEktronikksite*. Von TCA9548A- I2C Multiplexer: <https://wolles-elektronikkiste.de/en/tca9548a-i2c-multiplexer> abgerufen
- GeeksforGeeks. (5. August 2024). *Pulse Position Modulation (PPM)*. Von Pulse Position Modulation (PPM): <https://www.geeksforgeeks.org/pulse-position-modulation-ppm/> abgerufen
- JIMBLOM. (11. September 2024). *Sparkfun start something*. Von Voltage Divider: <https://learn.sparkfun.com/tutorials/voltage-dividers/all> abgerufen
- Pepperl+Fuchs. (13. September 2024). *Ultraschallsensor FAQ: Ausseneinflüsse auf die Sensorik*. Von <https://blog.pepperl-fuchs.com/de/2018/ausseneinfluesse-auf-ultraschallsensoren/> abgerufen
- Peterson, Z. (05. Oktober 2020). *Was ist eine Leiterplatte (PCB)?* Von <https://resources.altium.com/de/p/what-is-a-pcb> abgerufen
- PJRC. (5. August 2024). *PJRC Electronics Projects Components Available Worldwide*. Von <https://www.pjrc.com/store/teensy41.html> abgerufen

Random Nerd Tutorial. (15. September 2024). *Guide for TCA9548A I2C Multiplexer: Esp32, Esp8266, Arduino*. Von <https://randomnerdtutorials.com/tca9548a-i2c-multiplexer-esp32-esp8266-arduino/> abgerufen

Schultheiss, J. (13. September 2024). *Mehrfachreflexion*. Von <http://www.techniklexikon.net/d/mehrfachreflexion/mehrfachreflexion.htm> abgerufen

Tang, K. (31. Juli 2021). Beachlorarbeit. *Untersuchung und Verifizierung eines ToF-Sensors hinsichtlich seiner technischen Spezifikation*. Mittweida, Mittelsachsen, Deutschland. Von https://monami.hs-mittweida.de/frontdoor/deliver/index/docId/14248/file/BA_Ke_Tang.pdf abgerufen

Wikipedia. (13. September 2024). *Photodiode*. Von <https://en.wikipedia.org/wiki/Photodiode> abgerufen

Wikipedia. (13. September 2024). *Signal-Rausch-Verhältnis*. Von <https://de.wikipedia.org/wiki/Signal-Rausch-Verh%C3%A4ltnis> abgerufen

Deklaration

„Ich erkläre hiermit,

- dass ich die vorliegende Arbeit selbständig und nur unter Benutzung der angegebenen Quellen verfasst habe,
- dass ich auf eine eventuelle Mithilfe Dritter in der Arbeit ausdrücklich hinweise,
- dass ich eine allfällige Nutzung von Künstlicher Intelligenz (z.B. ChatGPT) in der Reflexion der Arbeit ausgewiesen und diskutiert habe,
- dass ich vorgängig die Schulleitung und die betreuende Lehrperson informiere, wenn ich diese Maturaarbeit, bzw. Teile oder Zusammenfassungen davon veröffentlichen werde, oder Kopien dieser Arbeit zur weiteren Verbreitung an Dritte aushändigen werde.“

Ort: Oberkirch LU, Schweiz

Datum: 10/10/24

Unterschrift:

